

TRABALLO FIN DE GRAO
GRAO EN ENXEÑARÍA INFORMÁTICA
MENCIÓN EN COMPUTACIÓN



Implementación dun pipeline de aprendizaxe profunda baseado en YOLO e CORAL-CNN para a automatización da avaliación de mordeduras en algas laminarias.

Estudante: Estevo Aldea Arias
Dirección: Manuel González Penedo

A Coruña, xuño de 2026.

Para o meu avó, Manolo. Espero chegar a ter unha fracción do teu valor.

Agradecementos

Grazas aos meus pais por apoiarme incondicionalmente. Grazas ás miñas irmás por ensinarme a ser unha boa persoa. Grazas á miña moza, María, por estar ao meu lado día tras día. Grazas ao resto da miña familia por confiar en min. Grazas aos meus amigos, por compartir momentos inesquecíveis xuntos, en concreto grazas a Coque por ser o meu irmán da Coruña, e a Ansur, Hugo, Mateo, Rubén, Javi e Alex por ser os mellores compañeiros.

Hiroto, thank you for opening the doors of your amazing country to me, our friendship will last forever. Thank you to James Scholz and Sebastian Pirie for giving me the determination to work hard to achieve my dreams.

Resumo

Historicamente, a bioloxía mariña dependeu da inspección visual de expertos para avaliar a saúde dos ecosistemas. No caso das algas laminarias (*Laminaria hyperborea*), a análise de depredación por peixes e invertebrados realízase mediante a asignación manual de notas nunha escala numérica de enteiros, un proceso subxectivo e custoso en tempo ante o crecente volume de datos visuais dispoñibles. Concretamente neste caso, non existen estudos previos que traten especificamente este tipo de alga. Ademais, existe un problema de preprocesado e normalización que solucionar nas imaxes, debido a que se tomaron dunha forma non estandarizada.

Este proxecto aborda a automatización do citado proceso mediante un pipeline de dúas etapas de aprendizaxe profundo. Utilízase un dataset de 4.984 imaxes proporcionado pola Facultade de Ciencias para adestrar, en primeira instancia, un modelo YOLO encargado da detección e localización das algas nas imaxes. Posteriormente, impleméntase unha arquitectura de regresión ordinal (CORAL-CNN) para clasificar o nivel de dano. En contraste coa clasificación tradicional, este enfoque preserva a orde lóxica da etiquetación por dano incremental, permitindo que o sistema funcione como unha ferramenta de soporte á decisión para os expertos, reducindo a subxectividade e optimizando o fluxo de traballo científico.

Abstract

Historically, marine biology has relied on expert visual inspection to assess ecosystem health. In the case of kelp (*Laminaria hyperborea*), the analysis of predation by fish and invertebrates is performed through the manual assignment of scores on an integer numerical scale, a subjective and time-consuming process given the growing volume of available visual data. Specifically in this case, there are no previous studies that specifically address this type of algae. Furthermore, there is an image preprocessing and normalization problem to resolve, as they were captured in a non-standardized manner. This project addresses the automation of the aforementioned process through a two-stage deep learning pipeline. A dataset of 4,984 images provided by the Faculty of Sciences is used to train, in the first instance, a YOLO model tasked with the detection and localization of the algae within the images. Subsequently, an ordinal regression architecture (CORAL-CNN) is implemented to classify the level of damage.

Unlike traditional classification, this approach preserves the logical order of incremental damage labeling, allowing the system to function as a decision-support tool for experts, reducing subjectivity, and optimizing the scientific workflow.

Palabras chave:

- YOLO
- CNN
- Visión por computador
- CORAL-CNN
- Alga Laminaria
- HPI
- IVR

Keywords:

- YOLO
- CNN
- Computer Vision
- CORAL-CNN
- Kelp
- HPI
- IVR

Índice Xeral

1	Introdución	1
1.1	Contextualización e xustificación	1
1.2	Achegamento do problema	2
1.3	Obxectivos	3
1.3.1	Obxectivo xeral	3
1.3.2	Obxectivos específicos	3
2	Estado da arte	4
2.1	Visión por computador aplicada á análise de imaxes biolóxicas	4
2.2	Aprendizaxe profunda para detección e clasificación	5
2.3	Regresión ordinal e estimación do dano	6
2.3.1	Regresión ordinal (CORAL-CNN)	7
2.4	Traballos relacionados	7
3	Materiais e metodoloxía	9
3.1	Conxunto de datos	9
3.2	Descrición das variables obxectivo	11
3.2.1	HPI	12
3.2.2	IVR	12
3.3	Enxeñaría de desenvolvemento	15
3.3.1	Avaliación de custos	18
4	Desenvolvemento da solución	20
4.1	Preprocesado de datos	20
4.1.1	<i>Script</i> de limpeza	20
4.1.2	Resultado da execución	21
4.1.3	Poda posterior e conclusión	21
4.2	Detección de alga en imaxes	22

4.2.1	Primeira Aproximación	22
4.2.2	Segunda Aproximación	23
4.3	Normalización de imaxes de algas	34
4.4	Arquitectura e adestramento da CNN	35
4.4.1	Preparación dos datos	35
4.4.2	Métricas empregadas	37
4.4.3	Elección de arquitectura	40
4.4.4	Optimización do modelo	43
4.4.5	Versión Final	49
4.5	Integración e funcionamento da <i>pipeline</i>	52
4.6	Resultados xerais da pipeline final	56
4.6.1	Resumo cuantitativo dos resultados	57
4.6.2	Resumo cualitativo dos resultados	57
5	Conclusións	61
5.1	Validación con expertos	61
5.1.1	Resumo da reunión	61
5.1.2	Resultados da proba	62
5.2	Conclusións do traballo	66
5.2.1	Cumprimento dos obxectivos	66
5.3	Limitacións do sistema	67
5.3.1	Erro en IVR	67
5.3.2	Erro en HPI	67
5.3.3	Adestramento orientado a lóxica experta	67
5.3.4	Limitacións do conxunto de datos e do preprocesado	68
5.4	Liñas futuras	68
5.4.1	Readiestramento con datos novos	68
5.4.2	Estandarización de toma de fotografías	69
5.4.3	Mellora da ferramenta de revisión	69
A	Repositorio do proxecto	71
	Glosario	73
	Bibliografía	75

Índice de Figuras

1.1	Exemplos variados de imaxes do <i>dataset</i> empregado.	2
2.1	Esquema xeral da arquitectura ConvNeXt descrita por Liu et al. [1].	5
2.2	Esquema xeral dunha arquitectura YOLOv8, pertencente á familia YOLO in- troducida por Redmon et al. [2].	6
3.1	Exemplos de variabilidade presente nas imaxes do <i>dataset</i> . Obsérvanse dife- renzas de iluminación, sombras, fondo non uniforme, elementos de referencia e anotacións visuais que condicionan o preprocesado posterior.	10
3.2	Diagrama xeral do proceso de desenvolvemento da solución.	17
3.3	Cronograma aproximado das principais fases de desenvolvemento do traballo.	18
4.1	Exemplos de recortes erróneos na primeira iteración de procesado do dataset.	24
4.2	Exemplos da interface para decidir se usar ou non o recorte automático no <i>dataset</i>	26
4.3	Evolución por épocas das perdas de adestramento e validación, así como das principais métricas de detección, durante o adestramento de YOLOv8n.	29
4.4	Comportamento das principais métricas de detección ao variar o umbral de confianza do modelo YOLOv8n.	30
4.5	Matriz de confusión do modelo YOLOv8n sobre o conxunto de validación. . . .	31
4.6	Distribución espacial e dimensional das caixas anotadas no <i>dataset</i> empregado para adestrar YOLO.	31
4.7	Mostras de predición do modelo YOLOv8n sobre o conxunto de validación. . .	33
4.8	Exemplos representativos do resultado da fase de normalización.	35
4.9	Transformación completa desde a imaxe orixinal ata a máscara binaria nor- malizada.	36
4.10	Exemplos de <i>heatmaps</i> empregados para analizar cualitativamente as rexións da imaxe nas que se apoia a CNN durante a inferencia.	40

4.11	Matriz de confusión de IVR para a execución con catro clases agrupadas. . . .	47
4.12	Diagrama de fluxo da <i>pipeline</i> de inferencia desenvolvida.	54
4.13	Exemplo dunha mostra marcada para revisión pola proximidade á fronteira HPI=0/1.	58
4.14	Exemplo dunha mostra clasificada correctamente pola <i>pipeline</i>	60

Índice de Táboas

3.1	Campos máis relevantes no ficheiro .CSV.	11
3.2	Partes anatómicas consideradas para a asignación do valor de HPI.	12
3.3	Escala de valores de HPI segundo o criterio experto.	13
3.4	Escala de valores de IVR segundo o criterio experto.	14
3.5	Estimación do custo de persoal segundo os perfís necesarios para o desenvolvemento.	19
4.1	Resultados finais do adestramento de YOLOv8n sobre o conxunto de validación.	29
4.2	Comparativa inicial entre os tres <i>backbones</i> probados na primeira formulación por regresión, tomando como referencia a mellor época de validación de cada un.	41
4.3	Comparativa resumida dos resultados en test para os tres <i>backbones</i> da primeira formulación por regresión.	42
4.4	Comparativa dos resultados en test entre as runs ordinais con saídas separadas e a run ordinal conxunta con <i>ConvNeXt_Tiny</i>	44
4.5	Comparativa en test entre a run ordinal conxunta sen ponderación e as dúas variantes con ponderación e <i>early stopping</i>	44
4.6	Comparativa en test entre a <i>baseline</i> con <i>ConvNeXt_Tiny</i> e as dúas probas de substitución por <i>ConvNeXt_Small</i>	45
4.7	Comparativa en test entre a <i>baseline</i> ordinal e a reformulación de IVR en catro clases agrupadas.	46
4.8	Comparativa entre a <i>baseline</i> ordinal e a primeira variante con IVR continuo condicionado pola predición de HPI segundo a avaliación reformulada.	49
4.9	Métricas principais da avaliación reformulada para a variante con IVR continuo condicionado por HPI.	50
4.10	Comparativa en test entre a <i>baseline</i> condicionada e a versión final.	52
4.11	Criterios de revisión manual segundo a distancia ás fronteiras críticas de HPI.	53

4.12	Motivos polos que unha mostra pode ser marcada para revisión manual pola <i>pipeline</i>	56
4.13	Exemplo do formato dos CSV xerados pola <i>pipeline</i> de inferencia.	56
4.14	Resumo operativo da execución final da <i>pipeline</i> sobre o subconxunto de validación con expertos.	57
4.15	Fila xerada pola <i>pipeline</i> para o exemplo de fallo.	59
4.16	Fila xerada pola <i>pipeline</i> para o exemplo de acerto.	59
5.1	Distribución do subconxunto de 200 imaxes empregado para a avaliación manual por parte dos expertos.	62
5.2	Variabilidade entre as dúas investigadoras e respecto da etiqueta orixinal. A métrica de IVR condicional calcúlase só cando as dúas avaliacións comparadas asignan unha nota positiva de IVR.	63
5.3	Comparación entre as métricas obtidas no test final e as obtidas polo sistema fronte á etiqueta orixinal no subconxunto revisado polos expertos.	64
5.4	Comparación do sistema coas dúas investigadoras fronte á variabilidade observada entre elas.	64
5.5	Variabilidade entre etiqueta orixinal, Sara e Nerea nos subconxuntos sinalados polos perfís de revisión. O MAE por pares calcula a diferenza media entre todos os pares dispoñibles no subconxunto, mentres que o desacordo de aplicabilidade de IVR mide diferenzas entre $IVR=0$ e $IVR>0$	65

Introdución

NESTA introdución contextualízase o traballo realizado mediante a presentación do problema abordado, dos obxectivos perseguidos e da metodoloxía xeral empregada. O propósito deste capítulo é ofrecer unha visión global do proxecto e establecer as bases dos capítulos seguintes, nos que se detallarán os aspectos técnicos e os resultados obtidos.

1.1 Contextualización e xustificación

Comprender e cuantificar os danos causados ás algas mariñas constitúe un paso esencial para a investigación e a conservación dos ecosistemas submariños. A análise deste tipo de danos permite avaliar o estado das propias algas e, ao mesmo tempo, obter información indirecta sobre a presión de depredación herbívora e sobre o comportamento alimentario das especies que interactúan con elas. Neste ámbito, especialistas en disciplinas como a bioloxía mariña deben examinar as mostras e determinar tanto a intensidade do dano observado como a súa natureza ou posible orixe.

Este labor realízase habitualmente de maneira manual, o que o converte nun proceso custoso en tempo e esixente desde o punto de vista técnico. O elevado volume de imaxes dispoñibles, xunto coa minuciosidade necesaria para efectuar unha valoración precisa, dificulta a análise eficiente dos datos. Ademais, a estimación do nivel de dano e a identificación do axente causante introducen un compoñente de subxectividade, especialmente nos casos intermedios, nos que unha mesma mostra pode recibir valoracións diferentes segundo o criterio da persoa experta que a examine.

Por outra banda, a documentación histórica xerada polos propios expertos, composta por rexistros de táboas e por material fotográfico, ofrece unha base de información valiosa para abordar a automatización deste procedemento. Neste escenario, a visión por computador e a aprendizaxe profunda preséntanse como ferramentas axeitadas para desenvolver un sistema de apoio á decisión capaz de asistir na avaliación do dano. A proposta deste traballo inscríbese

precisamente nesta liña, mediante o deseño dun pipeline orientado á análise de imaxes reais e á xeración de estimacións útiles como soporte ao labor experto.

1.2 Achegamento do problema

O problema que se aborda, tal e como se estableceu na contextualización, consiste en implementar unha solución que asista os expertos en bioloxía na cuantificación e clasificación de danos en algas laminarias. Para iso, dispoñemos dun *dataset* cedido polo grupo BIOCOST (Bioloxía Costeira)¹ da Facultade de Ciencias da UDC, no que se recollen fotografías que, aínda que comparten certos elementos comúns, como un plano cenital e un fondo semellante (mesa branca), non foron obtidas baixo un procedemento de captura completamente estandarizado. Existen imaxes que exceden os límites da mesa e deixan ver o fondo. Aparecen tamén, sombras derivadas dunha iluminación non controlada e a propia superficie de captura non é uniformemente branca sempre, xa que presenta unha franxa escura producida pola separación entre mesas. Ademais, nas imaxes, inclúense unha regra empregada como referencia de escala e identificadores numéricos asociados á captura.

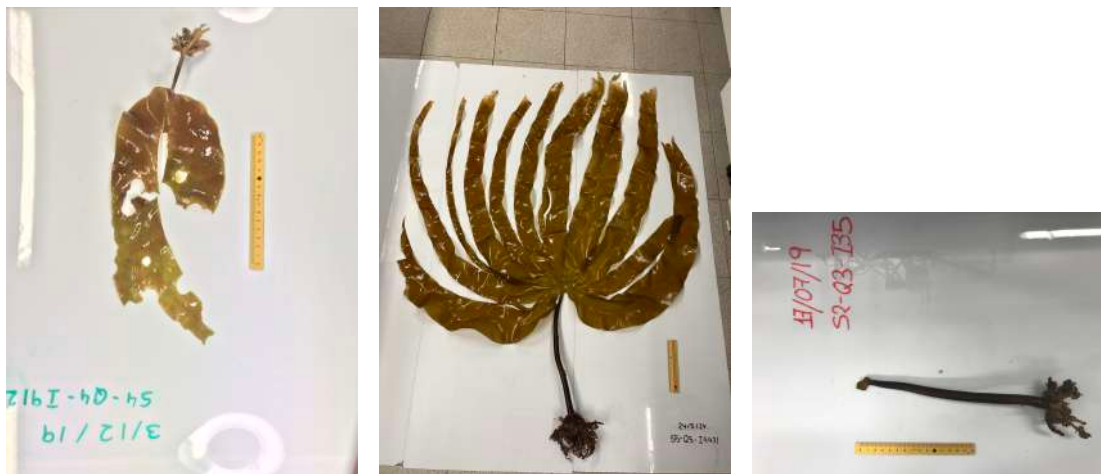


Figura 1.1: Exemplos variados de imaxes do *dataset* empregado.

Todo isto fai necesaria a división do sistema en varias etapas. En primeiro lugar, precísase un preprocesado das imaxes que permita estandarizar, na medida do posible, o *dataset* e reducir a influencia de variacións alleas ao fenómeno de interese, que poderían afectar ao comportamento da CNN. En segundo lugar, emprégase a propia CNN para realizar a inferencia sobre as imaxes tomadas polos expertos e determinar tanto o nivel de dano da alga como a orixe da depredación. De aquí en adiante, referirémonos a estes valores como HPI e IVR,

¹ Páxina web do grupo BIOCOST: <https://investigacion.udc.es/es/Research/Details/G000255>.

respectivamente. Esta é a nomenclatura empregada polos biólogos no *dataset* proporcionado.

Finalmente, resulta necesario deseñar unha *pipeline* que encapsule estes procesos e simplifique o uso do sistema por parte dos expertos. Deste xeito, os biólogos unicamente terán que incorporar unha imaxe ao programa, que lles devolverá a inferencia da CNN como apoio á súa toma de decisións.

1.3 Obxectivos

1.3.1 Obxectivo xeral

De forma global, este traballo pretende desenvolver un programa informático que poida asistir os biólogos na clasificación e cuantificación de danos causados a algas Laminarias, buscando un sistema que priorice a exactitude na inferencia e a facilidade de uso.

1.3.2 Obxectivos específicos

- Analizar e preparar o conxunto de datos dispoñibles para o problema.
- Deseñar unha etapa de preprocesado que reduza a variabilidade e normalice as imaxes atopadas no conxunto de datos.
- Desenvolver un modelo que proporcione unha inferencia para estimar as variables das algas (HPI, IVR).
- Integración do sistema nunha ferramenta usable polos expertos.
- Avaliar o comportamento do sistema con datos reais.

Estado da arte

Este capítulo recolle o marco teórico e os antecedentes necesarios para contextualizar a solución proposta neste traballo. En particular, revísanse os conceptos e técnicas máis relevantes relacionados coa visión por computador, a aprendizaxe profunda aplicada á análise de imaxes e os enfoques existentes para a avaliación automatizada do dano en organismos.

2.1 Visión por computador aplicada á análise de imaxes biolóxicas

A visión por computador é unha ferramenta de grande utilidade na análise automática de imaxes biolóxicas, ao permitir a extracción de información cuantitativa a partir de rexistros visuais de maneira máis rápida, reproducible e escalable que a inspección manual. Este tipo de técnicas resultan especialmente valiosas en contextos nos que o volume de datos é elevado ou nos que a avaliación visual require coñecemento experto, como sucede en tarefas de seguimento morfolóxico, clasificación de mostras ou detección de anomalías.

No ámbito acuático e mariño, esta tendencia tamén se fixo patente. O traballo de Kumar et al. [3] revisa avances recentes na análise de bioimaxes aplicadas á detección de deformidades esqueléticas en peixes de acuicultura e en especies empregadas en investigación biomédica, mostrando como a intelixencia artificial e os métodos modernos de análise de imaxe poden automatizar procesos tradicionalmente manuais. Aínda que o foco dese estudo non está nas algas nin na avaliación de dano por herbivoría, si evidencia unha tendencia xeral: a incorporación de técnicas de visión por computador en problemas biolóxicos permite mellorar a consistencia da análise e ampliar a capacidade de procesamento en escenarios con datos reais e heteroxéneos.

2.2 Aprendizaxe profunda para detección e clasificación

Dentro deste contexto, a aprendizaxe profunda adquiriu un papel central na análise de imaxes debido á súa capacidade para aprender representacións complexas directamente a partir dos datos. En particular, as redes neuronais convolucionais constitúen unha das arquitecturas máis empregadas en tarefas de clasificación visual, xa que permiten extraer patróns espaciais xerárquicos e adaptarse con boa eficacia a problemas nos que a aparencia, a textura ou a forma resultan determinantes.

Entre as arquitecturas convolucionais recentes, ConvNeXt destaca por reformular o deseño dunha CNN clásica incorporando decisións inspiradas nos *Vision Transformers*, mantendo ao mesmo tempo a natureza convolucional do modelo. Liu et al. [1] presentan esta familia como unha actualización das CNN para competir coas arquitecturas modernas baseadas en atención, o que a converte nunha alternativa relevante como extractor de características en tarefas de visión por computador.

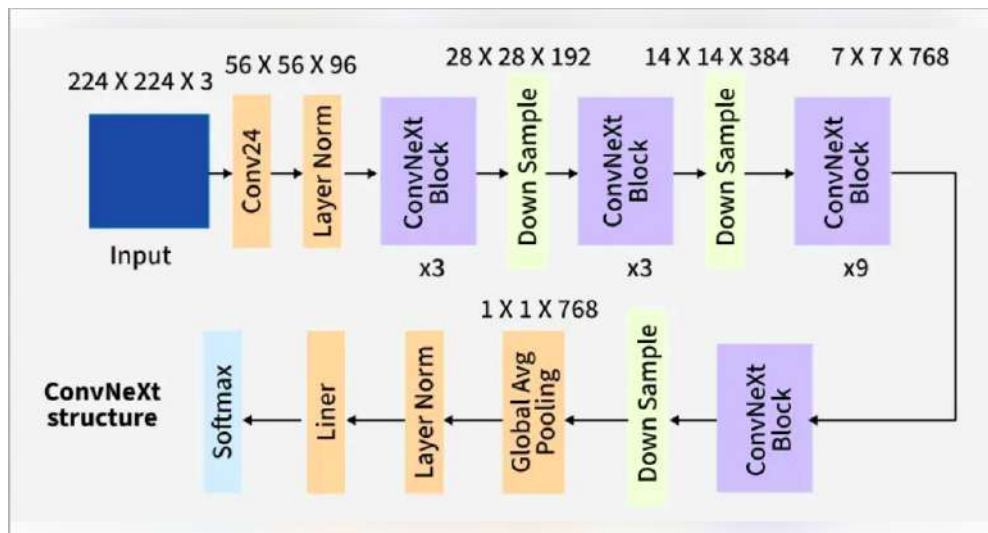


Figura 2.1: Esquema xeral da arquitectura ConvNeXt descrita por Liu et al. [1].

No caso da detección de obxectos, a familia de modelos YOLO, introducida por Redmon et al. [2], converteuse nunha das referencias máis influentes do campo. A revisión de Ramos e Sappa [4] analiza unha década de evolución de YOLO, desde as primeiras versións ata arquitecturas máis recentes, destacando como trazos característicos a súa eficiencia, a súa escalabilidade modular e a súa capacidade de adaptación a dominios diversos. Esta combinación de rapidez e precisión explica a súa ampla adopción en aplicacións de tempo real e en contornas nas que é necesario localizar automaticamente elementos de interese dentro da imaxe.

Para o presente traballo, este tipo de modelos resulta especialmente relevante porque a

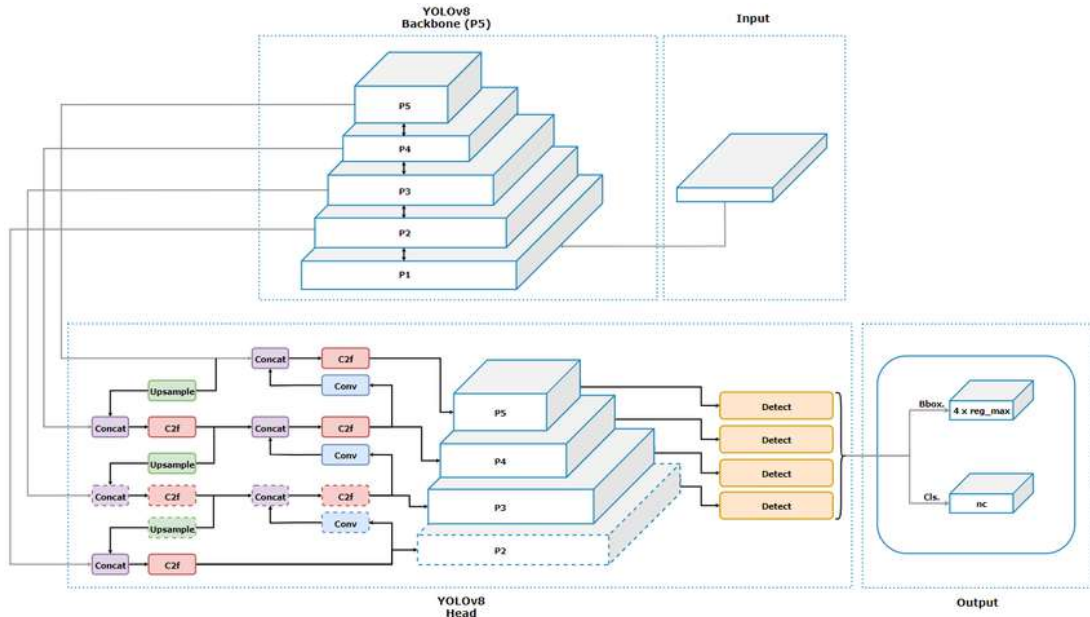


Figura 2.2: Esquema xeral dunha arquitectura YOLOv8, pertencente á familia YOLO introducida por Redmon et al. [2].

avaliación do dano non se realiza sobre unha rexión perfectamente recortada e estandarizada, senón sobre imaxes nas que primeiro é preciso identificar a zona correspondente á alga. Neste sentido, a utilización dun detector baseado en YOLO encaixa de maneira natural como primeira etapa do pipeline, ao permitir localizar o obxecto de interese antes de proceder á súa análise posterior.

2.3 Regresión ordinal e estimación do dano

En problemas nos que a variable obxectivo representa niveis ordenados, a formulación mediante regresión ordinal resulta máis axeitada que unha clasificación multiclase convencional, xa que ten en conta a relación de orde existente entre as etiquetas. No contexto deste traballo, esta característica é especialmente relevante, dado que o dano observado sobre as algas non se interpreta como un conxunto arbitrario de clases independentes, senón como unha progresión de intensidade.

Por este motivo, o uso de enfoques específicos de regresión ordinal permite modelar de maneira máis fiel a estrutura do problema e reducir erros conceptualmente máis graves, como tratar do mesmo xeito unha confusión entre niveis próximos e outra entre niveis afastados. Entre estes enfoques, CORAL-CNN [5] constitúe unha alternativa especialmente interesante pola súa capacidade para manter consistencia entre as predicións binarias que compoñen a saída ordinal.

2.3.1 Regresión ordinal (CORAL-CNN)

No traballo de Cao et al. [5] fórmulase a regresión ordinal como un problema no que as etiquetas non son simplemente categorías independentes, senón valores cunha orde relativa que debe ser tida en conta polo modelo. Os autores sinalan que os enfoques baseados en descompoñer esta tarefa en múltiples subtarefas de clasificación binaria poden producir inconsistencias entre predicións, o que resulta especialmente problemático cando se pretende estimar unha magnitude ordenada.

Como resposta a esta limitación, propónse **CORAL**, un *framework* de regresión ordinal para redes neuronais que garante consistencia no ranking e puntuacións de confianza monotónicas entre as distintas subtarefas. Ademais o artigo destaca que se trata dun método independente da arquitectura empregada, polo que pode adaptarse a modelos convolucionais comúns sen modificar a súa estrutura base. A avaliación experimental realizada polos autores no contexto da estimación de idade mostra unha redución do erro respecto de enfoques ordinais previos, o que reforza a súa utilidade en problemas nos que a orde entre etiquetas é un aspecto esencial.

2.4 Traballos relacionados

Os traballos previos máis próximos ao presente TFG poden agruparse en dúas liñas principais. Por unha banda, atópanse os estudos que empregan aprendizaxe profunda para resolver problemas de regresión ordinal en imaxes. Niu et al. [6] propuxeron un enfoque baseado nunha CNN con múltiples saídas para transformar a estimación da idade nun conxunto de subtarefas binarias correlacionadas, demostrando que a extracción de características e o modelado da orde poden aprenderse de forma conxunta. Sobre esta base, Cao et al. [5] formularon CORAL como unha mellora orientada a garantir consistencia entre as saídas ordinais, reforzando así a adecuación deste tipo de métodos para variables organizadas en niveis ordenados.

Por outra banda, existen traballos centrados na automatización da detección e da cuantificación da severidade en imaxes biolóxicas. O estudo de Sundar et al. [7] presenta un sistema para a detección de enfermidades en plantas e a estimación da súa severidade mediante unha combinación de clasificación con redes convolucionais e análise posterior da área afectada. Aínda que o dominio de aplicación é diferente, este enfoque resulta relevante porque comparte unha motivación moi próxima á deste TFG: reducir a subxectividade da avaliación visual experta e proporcionar medidas máis obxectivas e reproducibles do dano observado.

En conxunto, estes antecedentes mostran que xa existen ferramentas eficaces tanto para a localización de obxectos en imaxes como para a estimación de variables ordenadas ou de severidade en problemas biolóxicos. Con todo, a aplicación destas técnicas ao caso concreto da avaliación de mordeduras en algas laminarias segue sendo un problema sen explorar, espe-

cialmente cando se traballa con imaxes non estandarizadas e cunha formulación que require combinar detección, preprocesado e estimación ordinal nun mesmo fluxo.

Desde o punto de vista biolóxico, o traballo de Barrientos et al. [8] resulta imprescindible para este TFG. Sara Barrientos de la Llana, primeira autora dese estudo, pertencente ao grupo BIOCOST é unha das expertas que participou na reunión de validación descrita no Capítulo 5, analizou xunto con outros autores o colapso de bosques temperados de *kelp* no noroeste de España debido á herbivoría por peixes. O estudo mostra que as marcas de mordedura e a súa atribución a distintos grupos de herbívoros son relevantes para comprender a dinámica das poboacións de algas e a degradación dos arrecifes. A súa achega non é unha proposta de automatización mediante visión por computador, pero si proporciona unha motivación directa para o problema abordado neste traballo, que é dispoñer de ferramentas que axuden a cuantificar de maneira máis rápida e consistente o dano por herbivoría e a súa posible procedencia. Ademáis, a partir de este estudo xorde o *dataset* que se emprega neste traballo.

Materiais e metodoloxía

Neste capítulo descríbense os materiais empregados e a metodoloxía seguida para o desenvolvemento do sistema proposto. En particular, preséntase o conxunto de datos utilizado, os pasos de preprocesado aplicados ás imaxes, a estrutura xeral do pipeline deseñado e a configuración experimental empregada para avaliar o comportamento do sistema.

3.1 Conxunto de datos

O *dataset* empregado para este traballo foi cedido polo grupo BIOCOS (Bioloxía Costeira) da Facultade de Ciencias da UDC. Este conxunto componse de 4704 imaxes de algas *Kelp* / *Laminaria*, que, como xa se mencionou na sección de achegamento ao problema na introdución 1, presentan unha ampla variabilidade de captura. Esta heteroxeneidade afecta tanto ás condicións de iluminación e ao fondo como á presenza de elementos auxiliares e anotacións visibles na escena. Na Figura 3.1 móstranse varios exemplos representativos desta variabilidade. As imaxes tamén presentaban diferenzas de formato e tamaño, xa que se atoparon ficheiros en formatos .JPG, .JPEG, .PNG e .HEIC.

O *dataset* utilízase como base para o adestramento dos modelos que formarán parte do *pipeline* final. En particular, emprégase para adestrar un modelo YOLO orientado á detección das algas en imaxes capturadas polos expertos. Alén disto, o conxunto de datos serve para probar a normalización das imaxes antes do seu procesado pola CNN e, finalmente, para adestrar a propia CNN encargada de realizar a inferencia sobre as imaxes de entrada. Para este fin, o tamaño do *dataset*, malia a súa variabilidade, resulta axeitado para as necesidades do sistema proposto.

Adicionalmente, o conxunto de datos incluía un ficheiro en formato .CSV no que se recompilaba a información asociada a cada alga fotografada. Este ficheiro incorporaba campos de identificación, contexto temporal e espacial, medidas morfolóxicas e variables de avaliación biolóxica. Entre estes campos atopábanse, por exemplo, o identificador da fotografía (*Pho-*



(a) Reflexos, sombras, anotacións verdes e regra dourada.



(b) Fondo visible, liña de separación entre mesas e letras vermellas.



(c) Presenza dunha cinta métrica como referencia de escala.



(d) Iluminación diferente e anotacións en letras negras.

Figura 3.1: Exemplos de variabilidade presente nas imaxes do *dataset*. Obsérvanse diferenzas de iluminación, sombras, fondo non uniforme, elementos de referencia e anotacións visuais que condicionan o preprocesado posterior.

to_cod), a estación e a data de recollida, o lugar da mostraxe, distintas medidas biométricas e as variables de interese *HPI* e *IVR*. Porén, para o desenvolvemento deste traballo empregáronse unicamente os campos *Photo_cod*, *HPI* e *IVR*, ao seren os necesarios para relacionar cada imaxe coas etiquetas obxectivo do sistema.

Xunto con este ficheiro tamén se proporcionou o documento *Script for grazing marks.pdf*, no que se detallaba o significado das variables *HPI* e *IVR* e o procedemento seguido polos expertos para asignar unha puntuación a cada alga. Esta documentación resultou clave para interpretar correctamente as etiquetas dispoñibles e para establecer os criterios empregados no proceso de preparación dos datos, cuestión que se describe con máis detalle na seguinte sección.

Táboa 3.1: Campos máis relevantes no ficheiro .CSV.

Campo	Tipo	Descrición
Identificación e contexto de mostraxe		
<i>Photo_cod</i>	Identificación da fotografía	Identificador asociado a cada imaxe do conxunto de datos.
<i>Season</i>	Contexto temporal	Estación do ano na que se realizou a recollida.
<i>Date</i>	Contexto temporal	Data de captura ou recollida da mostra.
<i>Site</i>	Contexto espacial	Nome do lugar de mostraxe.
<i>Site cod</i>	Contexto espacial	Código abreviado do lugar de mostraxe.
<i>Species</i>	Información taxonómica	Especie rexistrada para a mostra.
Medidas biométricas e morfolóxicas		
<i>HPI</i>	Etiqueta ordinal de dano	Variable empregada para cuantificar o nivel de dano.
<i>IVR</i>	Etiqueta asociada á orixe da depredación	Variable empregada para caracterizar o tipo de depredación.

3.2 Descrición das variables obxectivo

Para interpretar correctamente as variables empregadas neste traballo utilizouse a documentación proporcionada polos expertos xunto co *dataset*, en particular o documento *Script for grazing marks.pdf*. Como xa se indicou, seleccionáronse unicamente *HPI* e *IVR* por seren

as variables máis relevantes para os obxectivos do sistema, isto é, a cuantificación do dano observado e a caracterización da súa posible procedencia. Nesta sección descríbense os criterios empregados polos biólogos para definir ambas as dúas variables, xa que constitúen as etiquetas obxectivo do modelo. e

3.2.1 HPI

HPI, ou *Herbivory pressure index* (Índice de presión de herbivoría), representa cuantitativamente o dano causado a unha alga por depredación herbívora. Segundo o criterio experto, a principal depredación destas plantas prodúcese maioritariamente por invertebrados, case sempre ourizos de mar, ou por vertebrados, principalmente peixes.

Para determinar o dano de forma precisa, os científicos dividen a alga en tres partes anatómicas. A Táboa 3.2 resume as seccións utilizadas durante a avaliación manual.

Código	Parte anatómica	Descrición
<i>H</i>	<i>Holdfast</i> (órgano de fixación)	Órgano de fixación da alga ao substrato.
<i>S</i>	<i>Stipe</i> (estipe)	Estrutura alongada que conecta a base coa parte foliar.
<i>B</i>	<i>Blade</i> (lámina)	Lámina principal da alga, onde se observa a maior parte das mordeduras.

Táboa 3.2: Partes anatómicas consideradas para a asignación do valor de HPI.

O valor de HPI depende da proporción de marcas de herbivoría observadas sobre a lámina (*blade*) e da conservación das distintas partes da alga. A escala empregada polos expertos recóllese na Táboa 3.3.

3.2.2 IVR

IVR, ou *Invertebrate/vertebrate ratio* (ratio de invertebrados/vertebrados), representa a proporción relativa entre marcas atribuídas a invertebrados e marcas atribuídas a vertebrados nas algas que presentan herbivoría observable. Esta variable utilízase, polo tanto, para caracterizar a posible procedencia do dano, e non a súa magnitude.

O valor de IVR só resulta interpretable nas mostras con HPI entre 1 e 4, é dicir, nas algas que conservan lámina e presentan marcas de herbivoría. Pola contra, as algas con HPI igual a 0 non presentan marcas, mentres que os niveis 5 e 6 indican unha perda total da lámina, o que impide determinar a procedencia das mordeduras. Nestes casos, o valor de IVR establécese como 0 por non ser aplicable.

HPI	Estado da alga	Interpretación
0	H + S + B (0 %)	Lámina completa sen marcas de herbivoría.
1	H + S + B (< 25 %)	Menos do 25 % da lámina presenta marcas.
2	H + S + B (25–50 %)	Entre o 25 % e o 50 % da lámina presenta marcas.
3	H + S + B (50–75 %)	Entre o 50 % e o 75 % da lámina presenta marcas.
4	H + S + B (75–100 %)	A maior parte da lámina presenta marcas de dano.
5	H + S	A alga conserva só o órgano de fixación e o estipe.
6	H	A alga perdeu tanto o estipe como a lámina.

Táboa 3.3: Escala de valores de **HPI** segundo o criterio experto.

A escala empregada polos expertos para interpretar esta variable recóllese na Táboa 3.4.

IVR	Proporción I/V	Interpretación
0	Non aplicable	Sen marcas observables ou sen lámina suficiente para determinar a procedencia.
1	0:100	Só se observan mordeduras atribuíbles a vertebrados.
2	5:95	Predominan claramente as mordeduras de vertebrados, con poucas marcas de invertebrados.
3	25:75	Obsérvanse máis mordeduras de vertebrados ca de invertebrados.
4	50:50	As marcas de vertebrados e invertebrados aparecen en proporción semellante.
5	75:25	Obsérvanse máis marcas de invertebrados ca de vertebrados.
6	95:5	Predominan claramente as marcas de invertebrados, con poucas mordeduras de vertebrados.
7	100:0	Só se observan marcas atribuíbles a invertebrados.

Táboa 3.4: Escala de valores de **IVR** segundo o criterio experto.

3.3 Enxeñaría de desenvolvemento

Desde o punto de vista da enxeñaría de desenvolvemento, o traballo organizouse como un proceso iterativo e incremental. Nun primeiro momento dividiuse o problema global nas mesmas grandes fases que estruturan esta memoria: análise do dominio e do conxunto de datos, preparación das imaxes, detección e recorte das algas, normalización dos recortes, deseño dos modelos de aprendizaxe profunda, avaliación experimental e integración final da *pipeline*. Esta división permitiu traballar sobre obxectivos máis acoutados e validar cada decisión antes de incorporala ao sistema completo.

O desenvolvemento non seguiu unha secuencia estritamente lineal. Aínda que as fases principais se presentan de maneira ordenada, en varias ocasións foi necesario volver sobre decisións previas a partir dos resultados obtidos. Por exemplo, a primeira análise dos recortes xerados automaticamente serviu para detectar erros de segmentación e motivou unha etapa posterior de revisión e curación dos datos. Do mesmo xeito, os resultados iniciais da *CNN* levaron a reformular a saída do modelo, comparar diferentes arquitecturas e introducir a lóxica experta asociada á aplicabilidade de *IVR*. Polo tanto, cada bloque do sistema foi refinándose mediante ciclos de implementación, avaliación e corrección.

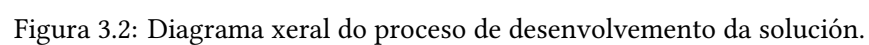
As primeiras semanas dedicáronse á comprensión do problema biolóxico, á revisión das variables incluídas no *dataset* e á identificación das dificultades presentes nas imaxes reais. A partir desta análise abordouse a preparación dos datos: primeiro mediante unha limpeza inicial e, posteriormente, mediante a detección automática da rexión ocupada pola alga con YOLO. Esta etapa permitiu separar o problema de localización do problema de estimación do dano, reducindo a variabilidade visual que tería que asumir directamente a rede de clasificación.

Unha vez obtidos os recortes, o esforzo centrouse na súa normalización e revisión. Esta fase foi especialmente relevante porque o conxunto de datos non procedía dun protocolo de captura completamente controlado, polo que existían diferenzas de iluminación, escala, orientación e fondo. Tras dispoñer dun conxunto de entrada máis homoxéneo, desenvóléronse as primeiras versións da *CNN*, preparáronse os particionamentos de adestramento, validación e proba, e comparáronse distintas formulacións do problema.

Na parte final do traballo, o desenvolvemento centrouse na mellora da estimación de *HPI* e *IVR*, na análise dos erros máis frecuentes e na integración da solución nunha *pipeline* de inferencia usable. Finalmente, realizouse unha validación con expertas sobre un subconxunto de imaxes, o que permitiu contrastar o comportamento do sistema co criterio humano e contextualizar as limitacións observadas.

Na Figura 3.2 represéntase de forma resumida o fluxo seguido durante o desenvolvemento da solución. O diagrama inclúe tanto as fases que forman parte da versión final coma algunhas

alternativas probadas e descartadas durante a evolución do sistema.



Cronograma aproximado do desenvolvemento

Outubro de 2025 – xuño de 2026

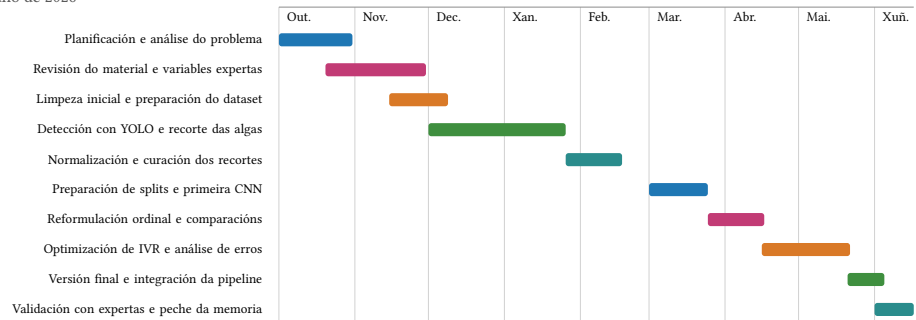


Figura 3.3: Cronograma aproximado das principais fases de desenvolvemento do traballo.

Na Figura 3.3 móstrase a planificación temporal aproximada seguida durante o proxecto. O solapamento entre varias tarefas reflicte precisamente o carácter iterativo do proceso: mentres unha fase se pechaba, os resultados obtidos condicionaban axustes nas fases anteriores ou servían de entrada para a seguinte iteración.

3.3.1 Avaliación de custos

A avaliación de custos realizouse a partir da planificación real do proxecto e das tarefas rexistradas no caderno de traballo. Segundo o cronograma da Figura 3.3, o desenvolvemento comezou o 1 de outubro de 2025 e rematou coa fase de validación e peche da memoria o 17 de xuño de 2026. Este intervalo comprende 260 días naturais. Se se exclúen fins de semana, obtéñense 186 días laborables. Considerando unha dedicación media de 4 horas por día laborable, a dedicación total estimada é de 744 horas.

Esta estimación non se obtén sumando directamente a duración de cada barra do cronograma, xa que varias fases se solapan. Por exemplo, a reformulación ordinal comezou antes de pechar completamente a comparación de modelos, e a integración final coincidiu parcialmente coa validación e coa redacción da memoria. Por este motivo, o cálculo toma como referencia os días laborables únicos do período global e reparte posteriormente ese esforzo entre os perfís necesarios.

As tarefas desenvolvidas condicionan os perfís considerados na estimación. O proxecto incluíu programación, limpeza e revisión dun conxunto de datos real, preparación de anotacións, adestramento dun detector YOLO, normalización de imaxes, comparación de varias arquitecturas CNN, reformulación do problema como regresión ordinal, análise de erros mediante *heatmaps* e integración da *pipeline* final.

Baixo unha hipótese de desenvolvemento profesional, a distribución estimada sería a seguinte. As tarifas horarias empregadas tomáronse como valores orientativos a partir de re-

ferencias públicas sobre tarifas medias por perfil profesional¹, axustándoas ao alcance académico e ao grao de responsabilidade de cada tarefa dentro do proxecto:

Perfil	Horas	Tarifa	Custo
Xestión e coordinación do proxecto	60	40 euros/h	2.400 euros
Análise e preparación de datos	170	25 euros/h	4.250 euros
Enxeñaría de aprendizaxe automática	360	30 euros/h	10.800 euros
Desenvolvemento e integración da <i>pipeline</i>	154	25 euros/h	3.850 euros
Total	744		21.300 euros

Táboa 3.5: Estimación do custo de persoal segundo os perfís necesarios para o desenvolvemento.

Ademais do custo de persoal, o proxecto require certos recursos técnicos. A parte de aprendizaxe profunda foi deseñada tendo en conta as limitacións dun equipo local con GPU NVIDIA RTX 3070, o que condicionou decisións como o uso de YOLOv8n, tamaños de *batch* reducidos, *mixed precision* en CUDA e a elección final de ConvNeXt_Tiny fronte a alternativas de maior tamaño. Tamén foi necesario dispoñer de almacenamento local para o *dataset*, os recortes, as imaxes normalizadas, os *checkpoints* e os resultados das execucións.

Non se imputan custos de licenzas, xa que o desenvolvemento se apoiou en ferramentas abertas ou de uso libre para este contexto, como Python, OpenCV, Ultralytics, PyTorch, Labelme e utilidades propias automatizadas mediante `Makefile`. Tampouco se consideran custos de servizos na nube, xa que os adestramentos se realizaron en local. Se se quixese trasladar esta estimación a un contorno profesional, podería engadirse unha pequena partida de amortización do equipo con GPU e almacenamento, estimada en arredor de 500 euros para o período de desenvolvemento.

Polo tanto, o custo principal estimado do proxecto é de 21.300 euros en persoal. Incluíndo unha imputación básica de recursos técnicos, o custo total aproximado ascendería a 21.800 euros.

¹ Clockify, tarifas horarias medias por profesión: <https://clockify.me/es/tarifas-horarias-medias>.

Desenvolvemento da solución

Neste capítulo descríbense os aspectos técnicos da solución desenvolvida, detallando as decisións de implementación adoptadas en cada etapa do sistema. En particular, abórdase o preprocesado dos datos, a detección das algas nas imaxes, a normalización das mostras, a arquitectura e o adestramento do modelo de inferencia e a integración final de todo o fluxo nun programa informático.

4.1 Preprocesado de datos

O primeiro paso da solución consistiu en depurar o *dataset* inicial para reducir a variabilidade introducida por erros ou inconsistencias básicas nos datos. Para iso desenvolveuse un *script* en Python, denominado `01_script_filtro.py`, orientado a simplificar o ficheiro de etiquetas, validar o formato das variables de interese e eliminar imaxes que non puidesen ser empregadas nas etapas posteriores do pipeline. Ademais da execución deste *script*, realizouse unha revisión posterior adicional baseada en criterios proporcionados polos expertos.

4.1.1 *Script* de limpeza

Entrada

O *script* parte de dúas fontes de datos definidas previamente no proxecto:

- O ficheiro Excel orixinal proporcionado polos expertos, no que se almacenaban as etiquetas asociadas ás mostras.
- A carpeta coas imaxes correspondentes ao *dataset*.

De maneira opcional, o programa admite o argumento `--dry-run`, que permite simular a limpeza das imaxes sen realizar modificacións sobre os ficheiros.

Procesado

En primeiro lugar, compróbase a existencia do ficheiro Excel e procédese á súa lectura mediante *pandas*. Unha vez cargado, selecciónanse unicamente as tres columnas relevantes para o problema abordado: `Photo_cod`, `HPI` e `IVR`. Esta redución permitiu traballar só cos metadatos estritamente necesarios para a tarefa de inferencia.

Posteriormente, as variables `HPI` e `IVR` convertéronse a formato numérico. Elimináronse as filas nas que algunha destas etiquetas non puido converterse correctamente ou non correspondía a un valor enteiro, xa que ambas representan niveis ordinais discretos. Deste xeito descartáronse valores non numéricos.

Como parte final desta etapa, revisouse a carpeta de imaxes para detectar ficheiros sen referencia válida no ficheiro das táboas. As imaxes cuxo identificador non aparecía en `Photo_cod` elimináronse, posto que non podían utilizarse no adestramento nin na avaliación do modelo.

Saída

Como resultado, xeráronse dous ficheiros de saída, en formato CSV e Excel, contendo exclusivamente as columnas `Photo_cod`, `HPI` e `IVR`. Esta etapa garantiu que as imaxes conservadas no repositorio dispoñían dunha referencia válida no ficheiro de etiquetas. A comprobación, porén, aplicouse só nese sentido. Elimináronse ficheiros de imaxe sen rexistro tabular asociado, pero non se eliminaron automaticamente as filas do ficheiro de táboas cuxa imaxe asociada non estivese presente na carpeta. Polo tanto, o ficheiro tabular filtrado podía conter máis rexistros ca imaxes dispoñibles. Esta limitación foi tida en conta durante a análise e retómase nas conclusións como posible mellora do pipeline.

4.1.2 Resultado da execución

Tras executar o *script* sobre o *dataset* inicial, composto por 5490 filas no ficheiro Excel e 4984 imaxes, obtívose o seguinte resultado:

- Elimináronse 4 filas con valores non numéricos en `HPI` ou `IVR` (por exemplo, `o`, `O` ou `NaN`).
- Elimináronse 7 imaxes por non teren correspondencia válida no ficheiro tabular.

4.1.3 Poda posterior e conclusión

Despois desta limpeza inicial, aplicouse unha poda adicional para asegurar o cumprimento das regras de etiquetado establecidas polos expertos. Esta revisión permitiu detectar 4 casos que incumprían ditos criterios, descritos na Sección-3.2, polo que foron eliminados tanto do conxunto de imaxes coma dos ficheiros finais de etiquetas.

En conxunto, esta fase de preprocesado permitiu partir dun conxunto de datos máis consistente e axeitado para o resto do traballo. Aínda que o volume de rexistros descartados non foi elevado, a limpeza resultou necesaria para evitar que erros de formato ou mostras sen utilidade condicionasen as etapas posteriores de detección, normalización e inferencia. Tras esta etapa, o conxunto final quedou composto por 4972 imaxes e 5486 rexistros tabulares.

4.2 Detección de alga en imaxes

4.2.1 Primeira Aproximación

Inicialmente procurouse unha solución baseada en técnicas clásicas de visión por computador empregando a librería OpenCV para Python en dous script novos, `02_recortar_algas.py` e `02_debug_recortar_algas.py`. O obxectivo era resolver a detección da alga cun enfoque relativamente simple e lixeiro, evitando nun primeiro momento o uso de modelos de aprendizaxe profunda. As distintas implementacións probadas compartían unha mesma idea central: aproveitar os contornos e as características visuais dos obxectos presentes na imaxe para identificar a alga de interese.

Esta primeira proposta partía de que as fotografías compartían un fondo maioritariamente branco, polo que, en principio, parecía factible segmentar os distintos elementos da escena mediante técnicas clásicas. Comezouse empregando o detector de bordes de Canny, un método que resalta cambios bruscos de intensidade na imaxe e permite identificar posibles contornos. A partir destes contornos tentouse localizar a alga combinando información sobre cor, posición e forma. Entre outros recursos, probase a detección dos números identificadores mediante filtrado por cor e a selección dos contornos cuxa aparencia se aproximaba máis á cor das algas, estimada previamente mediante inspección manual.

A primeira implementación ofreceu resultados moi limitados debido á elevada variabilidade do *dataset*. En moitos casos detectábanse regras como se fosen algas, especialmente cando eran pequenas; a cor da alga variaba notablemente en función da iluminación; e os números identificadores non sempre aparecían na mesma cor, o que impedía a súa eliminación mediante regras sinxelas. En conxunto, estes factores facían que o algoritmo resultase pouco robusto.

Co fin de mellorar esta primeira aproximación, introducíronse diversos parches orientados a eliminar elementos alleos á alga. No caso das regras, probouse primeiro a detección de contornos cunha relación de aspecto semellante á dunha regra e, posteriormente, o uso da transformada de Hough para detectar liñas rectas que permitisen identificalas. Aínda así, este procedemento seguía fallando con frecuencia, xa que a regra non sempre aparecía representada como un único contorno claro. Tamén se intentou eliminar contornos próximos ás liñas detectadas, pero isto provocaba a supresión de partes da propia alga cando ambas rexións

estaban moi próximas na imaxe.

Para os números identificadores empregáronse filtros de cor con distintas configuracións, pero comprobouse que a súa eficacia era reducida, sobre todo no caso do verde, que se confundía con facilidade coas tonalidades da alga. Tamén se probou a eliminación destas marcas a partir do tamaño da súa *bounding box*, pero esta estratexia descartaba igualmente algas pequenas.

Outra hipótese considerada foi asumir que a alga correspondía ao contorno máis centrado da imaxe. Con todo, esta condición tampouco se cumpría de forma suficientemente consistente, polo que a idea foi descartada. Así mesmo, experimentouse con técnicas baseadas en histogramas e segmentación automática, como o método de Otsu, que selecciona automaticamente un valor de corte para separar rexións de distinta intensidade. Isto tampouco produciu unha separación fiable entre a alga e o resto de elementos da escena.

Finalmente, tamén se intentou caracterizar unha suposta “forma de alga” a partir de medidas como a compacidade, entendida como a relación entre a área e o perímetro do contorno, ou a estrutura do propio contorno. Esta aproximación resultou pouco útil debido á gran variabilidade morfolóxica presente nas mostras.

As distintas ideas anteriores chegaron a combinarse nun sistema de puntuación ou “confianza”, no que cada contorno recibía unha valoración positiva ou negativa segundo a súa semellanza coa alga de interese. Porén, esta combinación incrementou notablemente a complexidade do código sen mellorar de maneira suficiente a robustez do método.

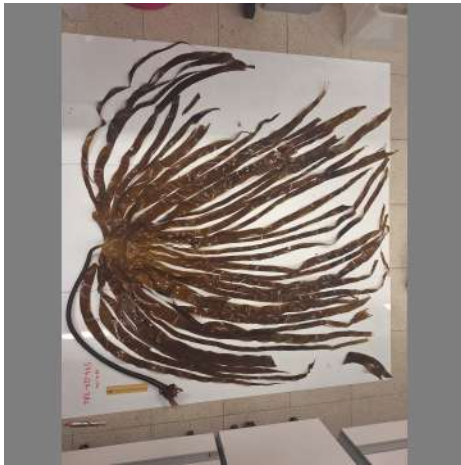
Como conclusión, esta primeira aproximación acabou xerando unha implementación relativamente extensa, de arredor de 500 liñas, e aínda así non ofreceu resultados satisfactorios. Aínda que nesta fase non se definiron métricas formais, xa que a validación se realizaba manualmente, estimouse que só arredor do 40 % do *dataset* se detectaba de maneira correcta. Algúns exemplos destes recortes erróneos móstranse na Figura-4.1. Á vista destes resultados, decidiuse abandonar este enfoque e optar por unha técnica menos sensible á variabilidade das imaxes, baseada en detección automática mediante aprendizaxe profunda.

4.2.2 Segunda Aproximación

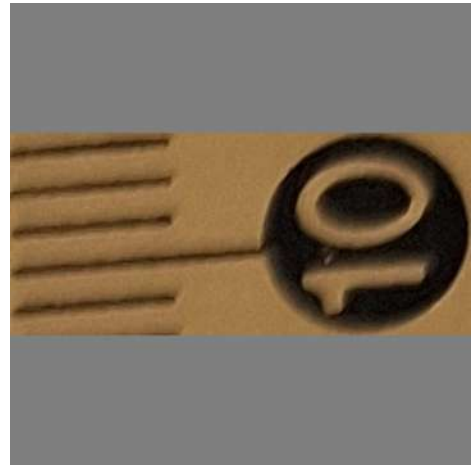
Preparación do *dataset*

Unha vez resultaron evidentes as limitacións da primeira aproximación, decidiuse buscar un método máis robusto fronte á elevada variabilidade do *dataset*. A alternativa escollida foi empregar YOLO, aproveitando a súa capacidade para a detección de obxectos en condicións máis heteroxéneas ca as que podían abordarse con regras clásicas de visión por computador.

A principal dificultade inicial deste cambio de enfoque non residía tanto no modelo en si como na preparación dun conxunto de adestramento axeitado. Para iso implementouse un



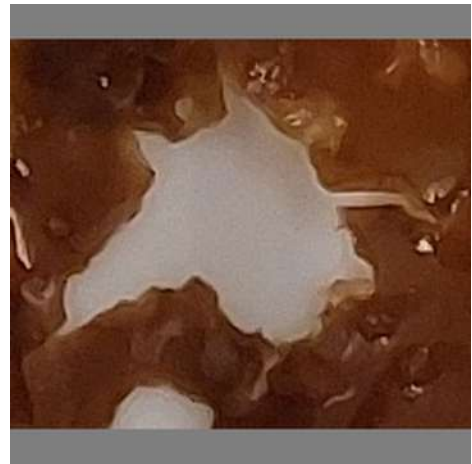
(a) Alga mal recortada por ter contacto cos bordes.



(b) Números da regra recortados como alga.



(c) Letra negra recortada como alga.



(d) Burato interior en alga recortado como a propia alga.



(e) Logo de regra recortado como alga.

Figura 4.1: Exemplos de recortes erróneos na primeira iteración de procesado do dataset.

novo *script* de Python, denominado `04_recortar_yolo.py`, orientado a xerar imaxes e anotacións nun formato compatible co adestramento de YOLO.

Este *script* reutilizaba a lóxica desenvolvida na primeira aproximación para propor, automaticamente, un *bounding box* candidato para a alga presente en cada imaxe. Dado que esta detección inicial non resultaba suficientemente fiable, incorporouse un modo adicional mediante o *flag* `--review`, que abría unha interface gráfica sinxela para revisar manualmente os recortes propostos. Nesta interface, que se mostra na Figura 4.2, visualizábase a imaxe orixinal xunto coa *bounding box* xerada automaticamente e decidíase se debía aceptarse ou descartarse.

Os controis da interface mantivéronse deliberadamente simples para non dificultar o proceso de revisión: `ESC` ou `q` para saír, `Enter` para aceptar o recorte e calquera outra tecla para descartalo. Deste xeito foi posible realizar unha curación manual dos candidatos xerados pola primeira aproximación e construír un conxunto de mostras válidas para a detección dunha única clase, definida como `CLASS_ID=0` e `CLASS_NAME='alga'`.

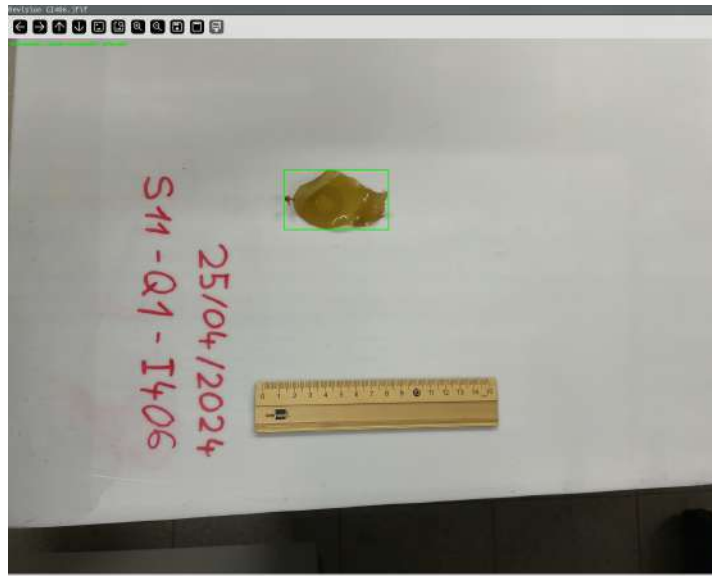
Ademais da revisión manual, o *script* xeraba automaticamente a estrutura de directorios requirida por YOLO, separando imaxes e etiquetas nos subdirectorios correspondentes a *train* e *val*. Así mesmo, convertía o formato das caixas delimitadoras ao esquema normalizado empregado por YOLO, isto é, (`x_center`, `y_center`, `width`, `height`). A división entre ambos os dous subconxuntos realizábase de maneira aleatoria segundo a proporción indicada polo *flag* `--val-split`. Xunto con estas saídas, xerábase tamén un ficheiro `data.yaml` coa información básica da clase necesaria para iniciar o adestramento do modelo.

Mediante este procedemento obtívose finalmente un conxunto de datos anotado de maneira suficientemente consistente para iniciar os experimentos con YOLO. A revisión manual resultou necesaria precisamente porque os recortes propostos pola primeira aproximación seguían presentando erros frecuentes, o que impedía a xeración automática dun conxunto fiable.

Despois de revisar manualmente arredor de 1300 imaxes, obtívose un *split* final composto por 636 imaxes de *train* e 146 de validación, correspondentes a unha partición aproximada do 80 % e 20 %, respectivamente. Aínda que este volume era reducido en relación co tamaño total do *dataset*, considerouse suficiente como punto de partida para o adestramento dun detector orientado á localización dunha única clase.

Elección de modelo YOLO

Unha vez preparado o *dataset* no formato requirido, procedeuse á elección do modelo YOLO que se empregaría nos experimentos. Esta decisión estivo condicionada por dous factores principais: por unha banda, a existencia dun número elevado de variantes recentes dentro da



(a) Interface do script cun recorte correcto, aceptaríaase manualmente.



(b) Interface do script cun recorte erróneo, rechazaríase manualmente.

Figura 4.2: Exemplos da interface para decidir se usar ou non o recorte automático no *dataset*.

familia YOLO e, por outra, as limitacións computacionais do contorno de traballo, restrinxido ao uso dun computador portátil equipado cunha tarxeta gráfica NVIDIA 3070.

A primeira decisión consistiu en seleccionar a versión de YOLO máis axeitada para o problema. O desenvolvemento desta familia de modelos acelerouse notablemente nos últimos anos, e na actualidade existen múltiples variantes documentadas pola propia plataforma de Ultralytics [9]. Entre as versións recentes consideráronse as seguintes:

- YOLOv8 (2023): O estándar da industria, consolidou YOLO como unha tecnoloxía de uso xeral.
- YOLOv9 (2024): Tratou de mellorar a preservación da información ao longo de redes profundas.
- YOLOv10 (2024): Reduciu a latencia eliminando a supresión de non máximos que se realizaba.
- YOLOv11 (2024): Introduciu melloras internas na extracción de características e na eficiencia do cómputo.
- YOLOv26 (2025): Busca mellora e estabilidade en CPUs ou hardware non dedicado.

A elección final foi YOLOv8, fundamentalmente por tres motivos. En primeiro lugar, tratábase dunha versión xa moi consolidada e ben documentada, o que facilitaba a reproducibilidade dos experimentos e a resolución de incidencias durante o desenvolvemento. En segundo lugar, o problema abordado nesta etapa reducíase á detección dunha única clase, polo que non resultaba imprescindible recorrer ás innovacións máis recentes da familia YOLO. Finalmente, malia que versións posteriores como YOLOv11 prometían certas melloras de eficiencia, considerouse que YOLOv8 ofrecía un equilibrio máis fiable entre madurez, soporte e custo de implementación para o contexto deste traballo.

A segunda decisión foi a selección do tamaño do modelo dentro da versión escollida. As variantes de YOLOv8 diferéncianse principalmente polo tamaño e a complexidade da rede neuronal [10]:

- n (Nano): Modelo máis pequeno e lixeiro.
- s (Small): Alternativa máis equilibrada entre custo e rendemento.
- m (Medium): Opción intermedia para tarefas máis esixentes.
- l (Large): Modelo de maior tamaño orientado a obter máis precisión.
- x (Extra Large): Variante máis profunda e máis custosa computacionalmente.

Tendo en conta as restricións de hardware e o feito de que o problema consistía unicamente na localización dunha clase, considerouse razoable comezar os experimentos coa variante máis pequena, YOLOv8n. A hipótese de partida era que un modelo lixeiro podería ofrecer un rendemento suficiente neste escenario, ao tempo que reduciría de maneira significativa o custo de adestramento e de inferencia.

Adestramento de YOLO

O adestramento implementouse no script `05_entrenar_yolo.py`. Este script actúa como unha interface mínima sobre a librería `Ultralytics`. Primeiro comproba que o ficheiro `data.yaml` exista, selecciona automaticamente o dispositivo de execución (GPU se está dispoñible e, en caso contrario, CPU), carga o modelo base indicado e delega o proceso en `model.train()`. Esta implementación compacta permitiu manter o experimento sinxelo sen introducir lóxica adicional innecesaria.

O experimento principal executouse coa seguinte configuración:

- `model=yolov8n.pt`: modelo preadestrado empregado como punto de partida.
- `data=yolo/data.yaml`: definición do *dataset* xerado na fase anterior.
- `epochs=60`: número total de épocas de adestramento.
- `imgsz=640`: resolución de entrada empregada polo detector.
- `batch=8`: tamaño de lote compatible coa memoria da GPU dispoñible.
- `device=0`: execución na primeira GPU do sistema.
- `project=runs_yolo, name=kelp4`: directorio e nome da execución gardada.

Ademais destes parámetros explícitos, `Ultralytics` aplicou a súa configuración por defecto para o resto dos hiperparámetros, incluíndo *optimizer* automático, *mixed precision* (`amp=true`), *patience* de 100 épocas e as transformacións de aumento de datos habituais para detección.

Os resultados finais do adestramento móstranse na Táboa 4.1. O mellor modelo gardado ao final da execución acadou valores moi elevados en todas as métricas de validación.

As curvas da Figura 4.3 amosan un comportamento estable ao longo das 60 épocas. As perdas de adestramento e validación descenden de maneira sostida, mentres que as métricas de precisión e *recall* soben rapidamente durante as primeiras iteracións e quedan practicamente saturadas arredor da época 30. En particular, o `mAP@50` pasa de 0.839 na primeira época a 0.995 na final, e o `mAP@50:95` sobe de 0.641 a 0.983. Isto indica que o modelo non só aprendeu a detectar a presenza da alga, senón tamén a axustar con bastante precisión as súas *bounding boxes*.

Métrica	Valor final
Precisión	0.99895
<i>Recall</i>	1.00000
mAP@50	0.99500
mAP@50:95	0.98335

Táboa 4.1: Resultados finais do adestramento de YOLOv8n sobre o conxunto de validación.

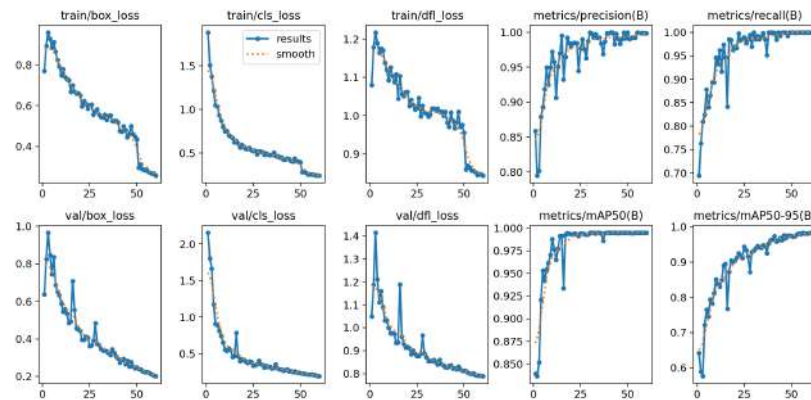
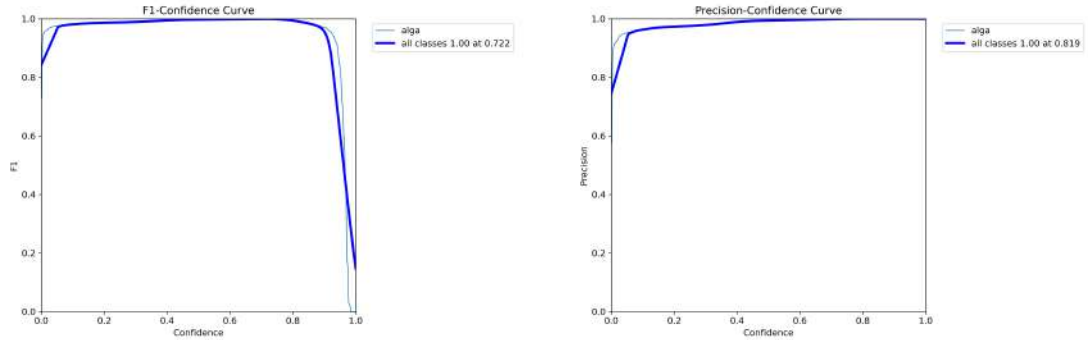


Figura 4.3: Evolución por épocas das perdas de adestramento e validación, así como das principais métricas de detección, durante o adestramento de YOLOv8n.

As curvas en función do umbral de confianza, recollidas na Figura 4.4, reforzan esta interpretación. A curva F1 mantense próxima a 1 nun intervalo amplo de umbrales e alcanza o seu máximo cunha confianza aproximada de 0.722, mentres que a precisión chega a 1.0 arredor de 0.819. Isto suxire que o detector resulta pouco sensible á elección exacta do umbral e que pode empregarse cun valor intermedio sen degradar de modo apreciable o equilibrio entre falsos positivos e falsos negativos.



(a) Curva F1 en función do umbral de confianza.

(b) Curva de precisión en función do umbral de confianza.

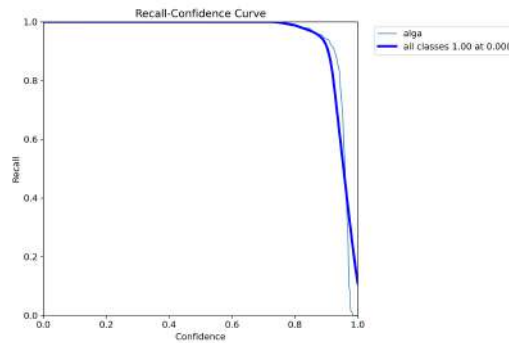
(c) Curva de *recall* en función do umbral de confianza.

Figura 4.4: Comportamento das principais métricas de detección ao variar o umbral de confianza do modelo YOLOv8n.

A matriz de confusión da Figura 4.5 confirma o bo comportamento observado nas métricas agregadas. Na avaliación de validación obtivéronse 131 deteccións correctas da clase *alga* e só 3 asociacións co fondo na configuración de avaliación empregada por Ultralytics. Pola súa parte, a Figura 4.6 mostra diversos tamaños de *bounding box*. Tamén vemos que a posición do centro da alga varía moito na imaxe, o que pode explicar os erros da primeira aproximación. Finalmente, na última figura podemos comprobar o tamaño en x e y das algas.

Por último, os exemplos cualitativos da Figura 4.7 permiten comprobar que o detector funciona correctamente en situacións variadas: algas de gran tamaño, exemplares pequenos,

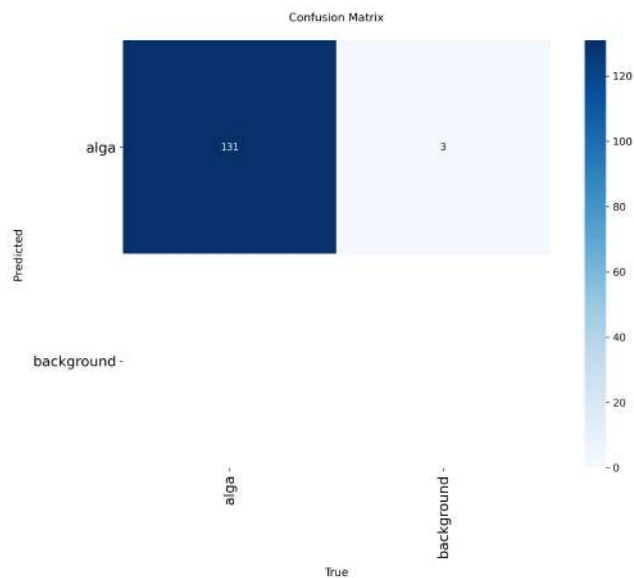


Figura 4.5: Matriz de confusión do modelo YOLOv8n sobre o conxunto de validación.

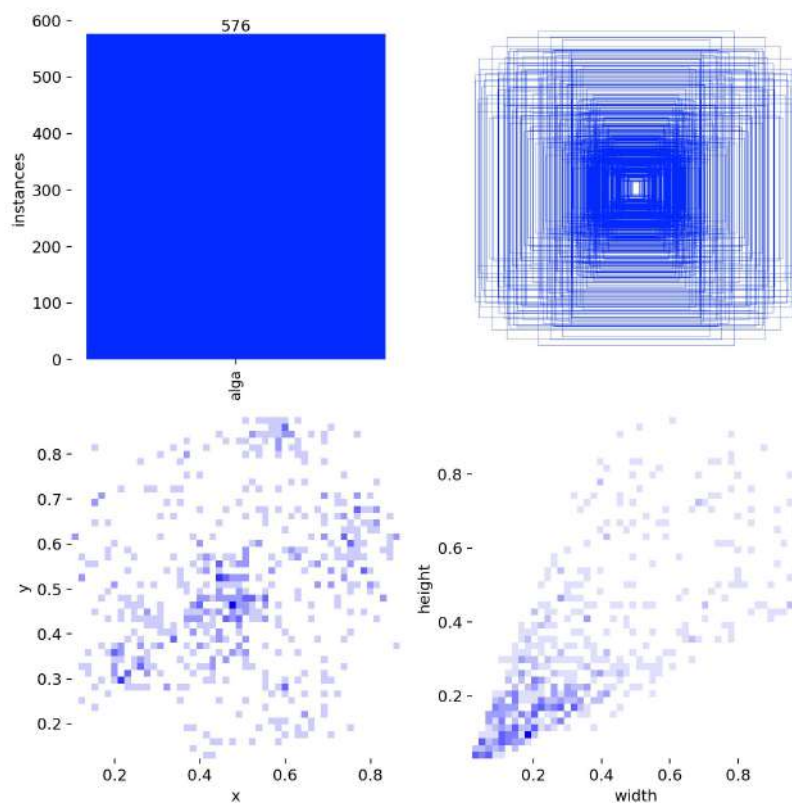


Figura 4.6: Distribución espacial e dimensional das caixas anotadas no *dataset* empregado para adestrar YOLO.

mostras orientadas vertical ou horizontalmente e imaxes nas que a regra ou as anotacións seguen visibles na escena. En conxunto, o modelo resultou suficientemente preciso como para ser incorporado como primeira etapa do *pipeline*, automatizando o recorte da rexión de interese antes da fase de normalización e inferencia.

Cómpre sinalar, ademais, varias limitacións específicas desta fase. A máis relevante é que non se reservou un conxunto de test independente para avaliar o detector, polo que os resultados presentados corresponden unicamente ao conxunto de validación e non permiten establecer unha medida definitiva da súa capacidade de xeneralización. Xunto con isto, o conxunto de validación empregado foi reducido e procede da mesma distribución de imaxes ca o adestramento, o que dificulta descartar que as métricas estean parcialmente favorecidas pola homoxeneidade das condicións de captura. Finalmente, ao tratarse dun modelo concibido como etapa de recorte dentro do *pipeline*, priorizouse que fose suficientemente robusto para automatizar a localización da alga, máis que realizar unha comparación exhaustiva entre múltiples variantes do detector. Por este motivo, os resultados deben entenderse como unha validación funcional da etapa de detección, e non como unha avaliación definitiva do problema en termos de xeneralización externa.

Inferencia de YOLO sobre o conxunto de datos

Unha vez se decidiu utilizar o YOLO adestrado, procedeuse a realizar unha inferencia con ese modelo sobre todo o *dataset*. Para facilitar esta tarefa, creouse un *script* novo que nomeamos `06_aplicar_yolo.py`.

Este *script* carga o modelo adestrado previamente e aplica a inferencia de YOLO sobre as imaxes seleccionadas do *dataset*. Para cada mostra execútase unha predición mediante `model.predict()`, e entre todas as deteccións devólvese unicamente a *bounding box* de maior confianza. A partir desa caixa, recórtase a rexión correspondente da imaxe orixinal e gárdase no directorio de saída co sufixo `_yolo.jpg`. O código permite configurar, ademais, o dispositivo de execución, a resolución de entrada, o limiar mínimo de confianza e o número de imaxes a procesar, xerando ao final un pequeno resumo co número de recortes gardados, imaxes sen detección e erros de procesado.

Na execución realizada sobre o conxunto completo de datos obtivéronse 4710 recortes gardados e 262 imaxes sen detección.

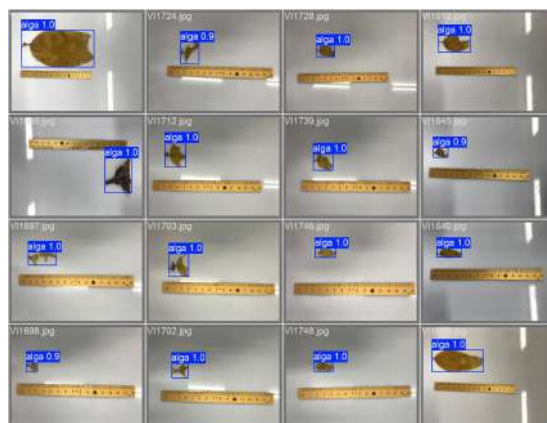
Executouse ese código para recortar todo o *dataset*. Neste punto xa se atopaba todo o conxunto recortado para a seguinte fase. Como revisión, decidiuse realizar unha pasada visual manual para comprobar se había algún recorte erróneo obvio. Elimináronse 60 imaxes, a maioría cun recorte fóra da alga. A aparición de non deteccións e recortes erróneos era esperable, debido ás limitacións do adestramento de YOLO que se mencionaron anteriormente. Nunha comprobación máis exhaustiva viuse que estas fotos eran especialmente problemáti-



(a) Exemplos con algas de tamaño medio e grande, con orientación variada.



(b) Predicións sobre mostras pequenas, onde o detector mantén caixas axustadas.



(c) Outros exemplos de validación con obxectos de pequeno tamaño e fondos semellantes.

Figura 4.7: Mostras de predición do modelo YOLOv8n sobre o conxunto de validación.

cas. Debido a isto e ao pequeno número de fotos erróneas, decidiuse continuar co mesmo YOLO e deixar como tarefa posterior unha guía para que os científicos saquen unhas fotos máis estandarizadas para evitar o fallo do YOLO. Como nota xeral, tamén é relevante mencionar que o sistema YOLO recortou algunha sombra e liñas centrais da mesa que poderían afectar ao posterior adestramento da CNN. Tras esta revisión manual, o conxunto final de recortes válidos quedou reducido a 4650 imaxes.

Deste xeito, o conxunto de recortes quedou preparado para a fase posterior de normalización das imaxes.

4.3 Normalización de imaxes de algas

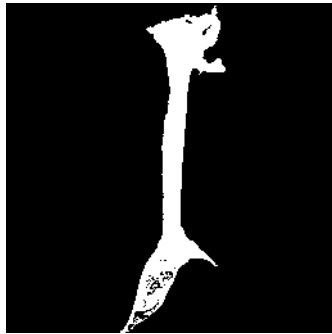
Unha vez se detectaron as algas dentro das imaxes do *dataset*, procedeuse a normalizalas para que poidesen constituír un *input* válido para a CNN a adestrar. Para realizar este proceso, implementouse un *script* máis, `07_normalizar_recortes.py`.

Este *script* parte dos recortes xerados na fase anterior e filtra unicamente aqueles cuxo identificador aparece no ficheiro CSV empregado como referencia. Sobre cada recorte aplícase un proceso de segmentación baseado no umbral de Otsu, calculado nas dúas polaridades posibles, e consérvase a máscara máis axeitada en función do compoñente conectado principal. A partir desta máscara binaria, a alga sitúase nun lenzo cadrado con fondo negro, mantendo a relación de aspecto, e reescálase posteriormente a un tamaño común para todo o conxunto. Deste xeito, a saída desta fase son máscaras binarias normalizadas e preparadas para a súa entrada na rede neuronal.

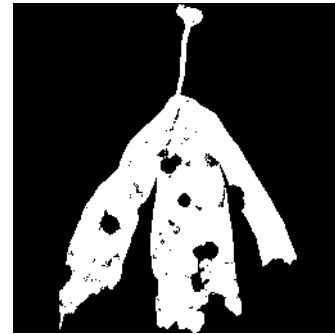
Cabe destacar que o *script* realiza dúas pasadas sobre o *dataset*. Na primeira, identifícase o recorte de maior tamaño, co obxectivo de definir o tamaño do lenzo cadrado común sobre o que se situarán posteriormente todas as mostras. Deste xeito, evítase unha redución prematura de resolución nesta fase, aínda que a entrada final da CNN sexa inferior por limitacións de hardware. Na segunda pasada, xéranse xa os recortes normalizados e colócanse efectivamente sobre o lenzo negro correspondente.

Para a execución, primeiro empregouse a flag `debug` para comprobar que a execución funcionaba. Unha vez se comprobou a correcta execución, normalizouse o *dataset* enteiro. A resolución das imaxes resultantes foi 4496x4496 píxeles. Ademais, atopáronse 5 imaxes sen referencia no CSV (esta comprobación debera terse feito na anteriormente, no preprocesado). Con isto, finalmente obtívose un *dataset* normalizado composto por 4645 imaxes como as da Figura 4.8.

Finalmente, a Figura 4.9 permite contextualizar o punto no que remata esta fase: a partir dunha imaxe orixinal do *dataset*, obtense finalmente unha máscara binaria normalizada e homoxénea, preparada para ser empregada como entrada na etapa seguinte. Unha vez com-



(a) Exemplo dunha alga normalizada con estipe grande.



(b) Exemplo dunha alga normalizada con ocos internos.

Figura 4.8: Exemplos representativos do resultado da fase de normalización.

pletado este proceso, o traballo céntrase xa no deseño da arquitectura da CNN e no seu adestramento para realizar a inferencia sobre as variables obxectivo.

4.4 Arquitectura e adestramento da CNN

Nesta sección explicárase todo o proceso de desenvolvemento da parte máis costosa de todo o traballo, a decisión arquitectónica e o adestramento da CNN encargada de realizar a inferencia dos valores HPI e IVR sobre as imaxes normalizadas. Isto débese a o aplo número de solucións que a tecnoloxía nos permite, dende a modificación de pesos durante o adestramento, ata o número de nodos de saída, pasando pola representación interna dos datos que tratamos de inferir. Por este motivo, veremos moitas variacións implementadas co obxectivo de buscar a solución que máis se axeite ao noso problema.

4.4.1 Preparación dos datos

O primeiro paso desta etapa foi construír o que se coñece como *manifest* do conxunto de datos. Neste contexto, o *manifest* é un ficheiro derivado do CSV filtrado no que se recollen unicamente as mostras válidas para a CNN, xunto co camiño da imaxe normalizada correspondente e cos valores obxectivo de HPI e IVR. A partir deste ficheiro realízase posteriormente a partición do *dataset* nas tres divisións habituais do adestramento dunha CNN: *train*, validación e test. As dúas primeiras empréganse durante o propio proceso de adestramento, mentres que a división de test se reserva para comprobar que o mellor modelo obtido, isto é, o correspondente á época con menor erro, realmente xeneraliza e mantén un bo comportamento sobre imaxes nunca vistas. Os valores que usaremos para comparar os distintos modelos que adestrems serán os resultantes da inferencia da CNN sobre esta última división das imaxes do *dataset*.

(a) Imaxe orixinal do *dataset*.

(b) Recorte proporcionado polo YOLO.



(c) Normalización do recorte.

Figura 4.9: Transformación completa desde a imaxe orixinal ata a máscara binaria normalizada.

Para isto implementouse un novo *script*, `08_build_split_manifest.py`, encargado de construír nunha única execución tanto o ficheiro *manifest* coma as particións de *train*, validación e test. O código percorre as imaxes normalizadas dispoñibles, relaciónaaas coas súas etiquetas no CSV filtrado mediante o identificador `Photo_cod` e xera un ficheiro tabular co camiño de cada imaxe e cos valores de HPI e IVR. A continuación, baralla aleatoriamente as mostrás cunha semente fixa e reparte o conxunto segundo as proporcións definidas para adestramento, validación e test, garantindo así unha partición reproducible.

A execución de este *script* proporcionou os seguintes resultados:

- *manifest*: 4645 imaxes.
- *train*: 3250 imaxes.
- validación: 697 imaxes.
- test: 698 imaxes.

4.4.2 Métricas empregadas

Para comparar os distintos adestramentos realizados, establecéronse varias métricas orientadas a avaliar a exactitude e a capacidade de xeneralización das CNN probadas. Non obstante, o conxunto de métricas empregado non permaneceu invariable ao longo do traballo. Nunha primeira etapa utilizáronse medidas globais, válidas para comparar de forma rápida distintas arquitecturas e configuracións xerais. Posteriormente, ao comprobar que a variable IVR presentaba un comportamento máis complexo e dependente de HPI, ampliouse o proceso de avaliación mediante un *script* específico de test, `10_test_cnn.py`, incorporando métricas máis alineadas coa interpretación biolóxica do problema. Exemplos da aplicación destas dúas familias de métricas aparecen nas comparacións de arquitecturas da Táboa 4.3 e, máis adiante, na avaliación reformulada da Táboa 4.9.

Métricas empregadas na avaliación inicial

Na fase inicial de experimentación empregáronse principalmente métricas globais, tanto durante o adestramento coma na avaliación posterior sobre o conxunto de test. Estas métricas permitiron comparar os primeiros modelos de maneira homoxénea e seleccionar as configuracións máis prometedoras.

Loss de adestramento e validación. A *loss* utilizouse como medida interna do proceso de optimización. A súa evolución ao longo das épocas permitiu comprobar se o modelo estaba aprendendo de forma estable e detectar posibles problemas de sobreaxuste. En particular, a *loss* de validación empregouse como criterio secundario para seleccionar o mellor punto de

adestramento cando varias execucións presentaban resultados similares nas métricas principais.

MAE. O *Mean Absolute Error* ou Erro Absoluto Medio cuantifica a distancia media entre as predicións do modelo e os valores reais. Durante esta primeira etapa foi a métrica principal de comparación entre modelos, xa que expresa directamente o erro medio en unidades da escala experta. No caso de modelos con dúas saídas empregáronse tres variantes:

- `mae_hpi`: MAE correspondente á predición de HPI.
- `mae_ivr`: MAE correspondente á predición de IVR.
- `mae_mean`: media de ambas magnitudes, utilizada como resumo global.

Estas métricas empréganse, por exemplo, nas comparacións das Táboas 4.3 e 4.10.

RMSE. O *Root Mean Squared Error* complementa ao MAE, xa que penaliza en maior medida os erros grandes. Isto resultou especialmente útil para detectar predicións afastadas do valor real, que no caso de IVR tiñan un impacto importante na interpretación biolóxica dos resultados. Do mesmo xeito ca no MAE, calculáronse `rmse_hpi`, `rmse_ivr` e `rmse_mean`. Un exemplo do seu uso para detectar erros afastados en IVR pode verse na Táboa 4.8.

Exactitude das predicións. Ademais dos erros medios, incorporáronse métricas discretas de acerto para avaliar en que medida as predicións coincidían exactamente coa nota esperada ou quedaban próximas a ela. En concreto, medíronse tres variantes:

- *exact match*: porcentaxe de mostras nas que a predición coincide exactamente coa etiqueta real.
- *within_1*: porcentaxe de mostras cun erro absoluto menor ou igual a 1.
- *within_2*: porcentaxe de mostras cun erro absoluto menor ou igual a 2.

Estas medidas permitiron matizar a información ofrecida por MAE e RMSE, distinguindo entre pequenos desaxustes e erros realmente afastados. Na versión final, por exemplo, o *within_1* condicional de IVR emprégase para valorar a proximidade da nota cando a variable resulta aplicable, como se recolle na Táboa 4.10.

Matrices de confusión. As matrices de confusión empregáronse para visualizar a distribución dos erros por clase e detectar patróns sistemáticos de confusión. Nunha primeira fase xeráronse matrices separadas para HPI e IVR, o que permitiu observar con claridade que os maiores problemas se concentraban na predición de IVR, especialmente nos erros entre niveis afastados. Un caso representativo desta análise móstrase na Figura 4.11.

Métricas incorporadas na avaliación posterior

A medida que avanzou o traballo, fíxose evidente que unha parte importante do erro en IVR non podía interpretarse illadamente, xa que esta variable só ten significado cando a predición de HPI indica que a avaliación de IVR resulta aplicable. Por este motivo, reformulouse a avaliación mediante métricas máis específicas, que pasaron a convivir coas métricas globais anteriores.

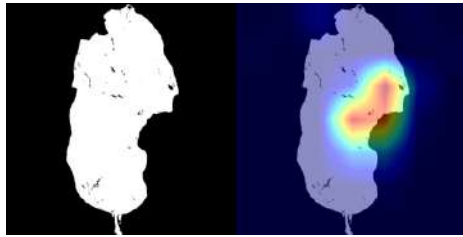
Aplicabilidade de IVR. Esta métrica formulouse como un problema binario: distinguir entre os casos nos que IVR non aplica ($IVR=0$) e aqueles nos que si debe existir unha nota válida ($IVR=1 \dots 7$). A avaliación realizouse mediante *accuracy*, *precision*, *recall*, *F1* e a súa correspondente matriz de confusión. O obxectivo desta medida foi separar os erros derivados de decidir se IVR debía aplicarse ou non dos erros asociados ao valor concreto da nota. Esta separación aparece aplicada nas Táboas 4.9 e 4.10.

Calidade da nota de IVR cando aplica. Unha vez separada a aplicabilidade, calculáronse de novo *MAE*, *RMSE*, *exact match*, *within_1*, *within_2* e matriz de confusión, pero unicamente no subconxunto de mostras nas que IVR realmente debía aplicarse e o modelo tamén predixo que aplicaba. Isto permitiu avaliar a calidade da nota de IVR sen mesturala cos casos en que o verdadeiro problema era decidir se a variable tiña sentido ou non. A Táboa 4.9 inclúe un exemplo desta avaliación condicional.

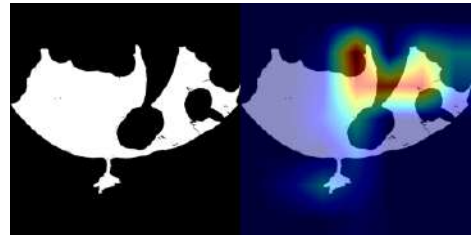
Consistencia interna entre HPI e IVR. Tamén se incorporou unha métrica de consistencia interna para comprobar se as predicións de HPI e IVR resultaban compatibles entre si. En particular, médironse os casos nos que o HPI predito implicaría que IVR non debería aplicarse, pero a saída do modelo asignaba igualmente un valor positivo, e viceversa. Esta métrica resultou útil para analizar as variantes máis recentes do sistema, nas que a coherencia entre ambas variables pasou a ser un aspecto relevante da avaliación. O seu efecto pode verse nas comparacións das Táboas 4.8 e 4.10.

Análise cualitativa mediante *heatmaps*

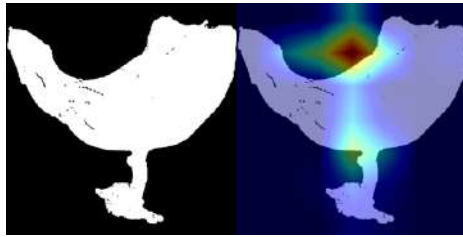
Por último, incorporouse unha análise cualitativa mediante *heatmaps* da CNN. Esta ferramenta non se empregou como métrica cuantitativa de comparación, senón como apoio interpretativo para visualizar en que rexións das imaxes se centraba o modelo ao realizar as súas predicións. A súa utilidade foi especialmente relevante nos casos problemáticos de IVR, xa que permitiu comprobar se a rede estaba atendendo ás mordeduras propiamente ditas ou, pola contra, a outras estruturas como o estipe, sombras ou artefactos do fondo. Algúns exemplos desta análise móstranse na Figura 4.10.



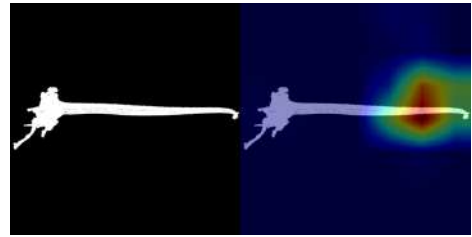
(a) Mapa de activación para unha predición de HP I.



(b) Mapa de activación nun caso con erro de HP I.



(c) Mapa de activación nun caso correctamente estimado.



(d) Mapa de activación nunha mostra con dano elevado.

Figura 4.10: Exemplos de *heatmaps* empregados para analizar cualitativamente as rexións da imaxe nas que se apoia a CNN durante a inferencia.

4.4.3 Elección de arquitectura

Chegados este punto, xa se tiña todo o necesario para adestrar unha CNN. Soamente se necesitaba escoller unha arquitectura para comenzar a facer probas. Isto implementouse nun *script* numerado como 09, que tivo distintos nomes que variaban acorde foi cambiando a arquitectura. O primeiro foi `09_train_regression.py`.

Solución por regresión

A primeira interpretación foi implementar unha CNN de regresión libre, isto significa que a súa saída non está condicionada por unha capa de saída *Softmax* que decida entre clases, sinon que a súa saída son números contínuos. Esta primeira interpretación decidiuse como base para escoller tanto o *Backbone* que empregariamos como a forma que usaríamos para chegar a inferencia da HPI e IVR, como dúas CNNs independentes, ou unha con dúas saídas.

Aínda que esta arquitectura dista moito da decisión final, esta primeira fase serviu para decidir facilmente aspectos da arquitectura, esperando que a CNN aprendese da forma máis exacta posible as regras dos expertos, e cunha primeira idea de usar decimais para buscar exactitude nas métricas, decidindo redondear cara o enteiro máis próximo no caso de que se usase como solución final.

Elección de un Backbone. Despois dunha primeira proba *smoke*, para comprobar o correcto adestramento dunha CNN de proba moi pequena no hardware dispoñible, pasouse a elixir entre 3 *backbones*:

- *EfficientNet-B0* [11]: Un modelo ultraeficiente que equilibra profundidade, ancho e resolución mediante *compound scaling* para maximizar a precisión co menor custo computacional posible.
- *ResNet18* [12]: Unha arquitectura clásica que introduce conexións residuais para evitar o deterioro do gradiente, permitindo un adestramento rápido, estable e eficaz.
- *ConvNeXt_tiny* [1]: Unha CNN moderna que redeseña as convolucións tradicionais inspirándose nos *Vision Transformers* para ofrecer unha alta precisión e un rendemento competitivo nun formato compacto.

Os tres adestramentos realizáronse baixo a mesma configuración experimental, modificando unicamente o *backbone* empregado en cada execución. En todos os casos utilizouse un tamaño de entrada de 224 píxeles, 30 épocas de adestramento, *batch size* 4, taxa de aprendizaxe de 10^{-4} , *weight decay* de 10^{-4} e función de perda baseada en MAE. Deste xeito, a comparación entre os tres modelos mantívose o máis controlada posible, deixando como única variábel principal a arquitectura de base.

Executáronse 3 adestramentos, nos que xa se intuía que a mellor solución sería *ConvNeXt_tiny*, cun *mae_mean* de 1.3034, como se resume na Táboa 4.2.

Backbone	Mellor época	mae_hpi	mae_ivr	mae_mean
<i>EfficientNet-B0</i>	epoch 18	0.6875	1.8467	1.2671
<i>ResNet18</i>	epoch 19	0.6859	1.9912	1.3386
<i>ConvNeXt_Tiny</i>	epoch 18	0.6782	1.8017	1.2340

Táboa 4.2: Comparativa inicial entre os tres *backbones* probados na primeira formulación por regresión, tomando como referencia a mellor época de validación de cada un.

Esta primeira vantaxe de *ConvNeXt_Tiny* quedou aínda máis clara cando se executou o *script* de test mencionado na sección anterior. A comparación xeral sobre o conxunto de test recóllese na Táboa 4.3, onde se observa que esta arquitectura mantén o mellor equilibrio global entre erro medio e exactitude.

Vendo o resultado de este test. Quedou claro que o mellor backbone era *ConvNeXt_Tiny*. Pero tamén se observou o principal problema da inferencia de HPI e IVR en xeral, que xa se mencionou brevemente, o erro de IVR é moito maior e máis disperso que o de HPI. Ao repasar os criterios dos expertos, este problema está claro, a CNN non parecía captar de maneira

Backbone	mae_mean	mae_hpi	mae_ivr	within1_mean
<i>EfficientNet-B0</i>	1.4058	0.7636	2.0479	0.6042
<i>ResNet18</i>	1.4560	0.7552	2.1568	0.5981
<i>ConvNeXt_Tiny</i>	1.3034	0.6418	1.9651	0.6827
Backbone	rmse_mean	rmse_hpi	rmse_ivr	
<i>EfficientNet-B0</i>	2.2077	1.4681	2.9472	
<i>ResNet18</i>	2.0945	1.1699	3.0190	
<i>ConvNeXt_Tiny</i>	2.0928	1.1033	3.0823	

Táboa 4.3: Comparativa resumida dos resultados en test para os tres *backbones* da primeira formulación por regresión.

consistente a lóxica experta que determina cando IVR debe tomar o valor 0. Ademais, como IVR depende directamente do resultado de HPI, tamén se estaría arrastrando o erro no caso de aplicar a lóxica experta. Aínda así, este primeiro adestramento demostrou que a CNN non estaba tan afastada dos valores esperados, e que o enfoque baseado en CNN resultaba prometedore.

Cómpre aclarar que, nesta primeira formulación por regresión, a CNN producía valores continuos para HPI e IVR. Porén, dado que as etiquetas expertas do problema son discretas e enteiras, nas avaliacións de test estas predicións aproximáronse ao enteiro máis próximo antes de calcular as métricas correspondentes. Esta decisión permitiu comparar a saída da rede coa escala de notas empregada polos expertos, aínda que tamén puxo de manifesto a limitación de base desta formulación fronte a un problema de natureza ordinal.

Solución por clasificación ordinal

Co resultado obtido, quedou claro que era necesario cambiar a aproximación e buscar unha solución que tratase o problema como unha clasificación ordinal que replicase de mellor forma o criterio experto empregado para avaliar as algas.

Esta decisión estivo motivada pola propia natureza das variables obxectivo. Tanto HPI como IVR se expresan mediante etiquetas enteiras e ordenadas, non mediante magnitudes continuas. A formulación por regresión empregada inicialmente obrigaba a producir saídas continuas e a aproximalas posteriormente ao enteiro máis próximo, introducindo unha representación que non se axustaba ben á escala real empregada polos expertos. Pola contra, unha formulación ordinal permite aprender directamente as fronteiras entre niveis consecutivos da escala e, ademais, penaliza de maneira máis natural os erros afastados, por exemplo confundir unha mostra etiquetada como 1 cunha predición de 5, que os erros próximos como 1 fronte a 2. Este comportamento encaixa mellor coa interpretación biolóxica do problema, na que non

todos os erros teñen a mesma gravidade.

Como vimos, utilizouse o estudo sobre CORAL-CNN [5] como base do noso código, pero non se aplicou exactamente como se propón no *paper*. No traballo orixinal, CORAL preséntase como un mecanismo para adaptar unha CNN convencional a un único problema ordinal, substituíndo a cabeza de clasificación estándar por unha saída formada por $K-1$ tarefas binarias acumulativas asociadas aos umbrais entre clases consecutivas. A predición final non se obtén escollendo unha categoría entre K posibilidades, senón reconstruíndoa a partir dese conxunto ordenado de decisións binarias. Na nosa implementación mantívose esa mesma idea de base, isto é, combinar un *backbone* convolucional cunha cabeza ordinal, pero adaptándoa a un escenario con dúas variables obxectivo. Así, o *backbone* compartido segue a encargarse da extracción de características visuais das imaxes, mentres que a capa final se dimensiona para producir os umbrais necesarios de HPI, de IVR ou de ambas variables á vez. A diferenza fundamental con respecto ao escenario do artigo non está, polo tanto, na lóxica ordinal da saída, senón no feito de que aquí se estende a unha formulación multisaída máis acorde co problema real do traballo.

Elección de saída da CNN. Unha vez implementada a clasificación ordinal, e como xa adiantamos, decidiuse realizar unha comparación entre usar dúas CNN independentes (unha para HPI e outra para IVR) ou soamente unha CNN con dúas saídas.

Despois de executar os tres adestramentos e avaliar cada unha das súas mellores runs sobre o conxunto de test, observouse que a opción con dúas saídas compartidas resultaba a máis sólida en termos xerais. A run conxunta mellorou simultaneamente o `mae_hpi` e o `mae_ivr` respecto das runs especializadas, acadando un `mae_mean` de 1.2142 fronte a 1.9656 no caso de IVR por separado e 0.5745 no caso de HPI por separado. Ademais, o acerto exacto e o acerto dentro dun erro absoluto menor ou igual a 1 tamén foron competitivos, para HPI, a run conxunta obtivo mellor *exact match* ca a run especializada, mentres que para IVR tamén mellorou o *within_1*. Estes resultados suxiren que compartir un mesmo *backbone* para ambas variables permitía aproveitar mellor a información visual común presente nas imaxes, polo que se decidiu continuar o desenvolvemento a partir desta configuración multisaída.

4.4.4 Optimización do modelo

Ponderación e *Early Stopping*

Unha vez establecida a configuración ordinal con dúas saídas compartidas, o seguinte paso consistiu en modificar a función de perda para introducir unha ponderación explícita entre HPI e IVR. A implementación realizouse sobre a perda ordinal binaria acumulativa de ambas variables, esta pasou a calcularse como a suma ponderada da *loss* ordinal de HPI e da de IVR, empregando dous coeficientes `w_hpi` e `w_ivr` normalizados. Deste xeito, puido

Configuración	Saída	mae	rmse	exact_match	within_1
CNN separada	HPI	0.5745	1.1119	0.5946	0.9212
CNN separada	IVR	1.9656	3.2211	0.4742	0.6433
CNN conxunta	HPI	0.5630	1.1412	0.6261	0.9112
CNN conxunta	IVR	1.8653	3.1199	0.4986	0.6519
CNN conxunta	media	1.2142	2.1305	0.5623	0.7815

Táboa 4.4: Comparativa dos resultados en test entre as runs ordinais con saídas separadas e a run ordinal conxunta con *ConvNeXt_Tiny*.

asignárselle máis peso á parte de IVR, que xa se identificara como a máis difícil, sen alterar a arquitectura nin a lóxica xeral do modelo.

Sobre esta base incorporouse ademais un mecanismo de *early stopping*, co obxectivo de detectar o punto no que a validación deixaba de mellorar e evitar adestramentos máis longos do necesario. Para probar a efectividade destes cambios realizouse unha primeira execución que asemellaba o *baseline* anterior, pero xa con *early stopping* con paciencia 8 e pesos efectivos $w_{hpi}=0.4$, $w_{ivr}=0.6$. A continuación realizouse unha segunda proba, na que se reforzou lixeiramente a importancia de IVR mediante a combinación $w_{hpi}=0.3$, $w_{ivr}=0.7$, mantendo a mesma filosofía de adestramento e a mesma arquitectura base.

O efecto desta modificación foi moderado pero consistente. En validación, as dúas execucións acadaron os mellores resultados nas primeiras épocas, o que confirma a utilidade do *early stopping*: a primeira alcanzou o seu mellor mae_mean na época 6 ($mae_mean=1.1083$), mentres que a run 9 o fixo na época 5 ($mae_mean=1.1105$). En ambos os dous casos, as últimas épocas gardadas presentaron peores valores de validación, polo que o criterio de *early stopping* evitou prolongar innecesariamente o adestramento. A comparación máis relevante, non obstante, foi a realizada sobre o conxunto de test, tomando como referencia a run ordinal conxunta sen ponderación previa.

Execución	mae_mean	mae_hpi	mae_ivr	rmse_ivr	within_1_mean
Sen ponderación	1.2142	0.5630	1.8653	3.1199	0.7815
Pesos 0.4/0.6	1.2120	0.5802	1.8438	2.9878	0.7772
Pesos 0.3/0.7	1.1908	0.5567	1.8250	3.1041	0.8087

Táboa 4.5: Comparativa en test entre a run ordinal conxunta sen ponderación e as dúas variantes con ponderación e *early stopping*.

Como se observa na Táboa 4.5, a introdución de pesos produciu melloras pequenas pero relevantes. A configuración 0.4/0.6 apenas modificou o erro medio global respecto da orixinal, pero reduciu o $rmse_ivr$, o que suxire unha diminución dalgúns erros máis afastados

nesta variable. Pola súa parte, a configuración 0.3/0.7 foi a que ofreceu o mellor comportamento global: reduciu tanto `mae_mean` como `mae_ivr` con respecto ás dúas referencias anteriores e, ademais, mellorou o acerto medio dentro de ± 1 punto. Aínda que o `rmse_ivr` aumentou lixeiramente respecto a outra execución ponderada, o conxunto dos resultados indica que esta configuración acadou o mellor equilibrio entre precisión global e capacidade para priorizar IVR. Por este motivo, a configuración de pesos 0.3/0.7 pasou a considerarse a nova *baseline* do traballo para as optimizacións posteriores.

Mellora de backbone

Unha vez fixada como *baseline* a configuración ordinal con pesos 0.3/0.7 sobre *ConvNeXt_Tiny*, a seguinte hipótese de traballo foi que unha arquitectura de maior tamaño podería capturar mellor os patróns visuais asociados ás mordeduras e reducir así os erros máis afastados, especialmente en IVR. Para comprobar esta posibilidade substituíuse o *backbone* por *ConvNeXt_Small*, mantendo o resto da configuración. Isto deu lugar a dúas execucións: unha primeira proba con *batch size* 4 e unha segunda repetición con *batch size* 8, ao considerarse que o primeiro resultado negativo podía estar condicionado pola redución do *batch size*.

Os resultados non confirmaron a hipótese inicial. Con *batch size* empeorou claramente o resultado da *baseline*, sobre todo por un aumento do erro en IVR. A repetición con *batch size* 8 mellorou parcialmente este comportamento e achegouse aos resultados da *baseline*, pero sen superala de maneira consistente. En consecuencia, o cambio de *backbone* non compensou nesta fase do traballo e mantívose *ConvNeXt_Tiny* como arquitectura de referencia.

Configuración	<code>mae_mean</code>	<code>mae_hpi</code>	<code>mae_ivr</code>	<code>rmse_ivr</code>	<code>within_1_mean</code>
<i>Baseline ConvNeXt_Tiny</i> 0.3/0.7	1.1908	0.5567	1.8250	3.1041	0.8087
<i>ConvNeXt_Small</i> <i>batch</i> =4	1.2858	0.5616	2.0100	3.2231	0.7787
<i>ConvNeXt_Small</i> <i>batch</i> =8	1.2120	0.5874	1.8367	3.0133	0.7830

Táboa 4.6: Comparativa en test entre a *baseline* con *ConvNeXt_Tiny* e as dúas probas de substitución por *ConvNeXt_Small*.

Ademais, a análise das matrices de confusión puxo de manifesto que seguían aparecendo erros moi afastados en IVR, por exemplo casos reais de IVR=7 preditos como 0 ou 1, así como mostras con IVR baixo que eran inferidas como valores altos. Estes fallos prexudicaban especialmente o `rmse_ivr` e non podían interpretarse sempre como un problema illado da saída de IVR, xa que en parte viñan arrastrados por erros previos na inferencia de HPI. Por este motivo, nas iteracións posteriores do traballo tratouse de abordar o problema

non só mediante cambios de *backbone*, senón tamén reformulando a avaliación e a propia representación de IVR.

Agrupación de IVR

Despois de comprobar que aumentar o tamaño do *backbone* non solucionaba os erros afastados en IVR, probouse unha reformulación da propia variable. A hipótese desta proba era que a escala completa de IVR, con oito valores posibles, podía estar contribuíndo ao erro polo número de niveis, ao forzar distincións finas e parcialmente subxectivas entre categorías próximas. Isto podía facer que os datos de adestramento non achegasen unha separación visual suficientemente consistente para inferir IVR con exactitude. Por este motivo, mantívose a arquitectura de referencia con *ConvNeXt_Tiny*, saída ordinal conxunta e pesos 0.3/0.7, pero substituíuse a saída de IVR por catro clases agrupadas: 01, 23, 45 e 67.

Esta modificación reducía a dificultade formal da tarefa de IVR, xa que a rede deixaba de ter que distinguir entre todos os niveis individuais e pasaba a predicir intervalos máis grosos. Con todo, esta redución tamén obriga a interpretar as métricas con cautela: un *mae_ivr* obtido sobre catro clases non é directamente comparable cun *mae_ivr* calculado sobre a escala orixinal de oito clases. Por iso, ademais do valor bruto, analizouse o erro normalizado polo rango ordinal e tamén unha comparación na que as predicións da *baseline* se colapsaron ao mesmo esquema de catro grupos.

Configuración	Escala de IVR	<i>mae_ivr</i>	<i>mae_ivr</i> normalizado	<i>Exact match</i> agrupado
<i>Baseline</i> 0.3/0.7	0-7	1.8324	0.2618	–
<i>Baseline</i> colapsada	01/23/45/67	0.8009	0.2670	0.6533
IVR agrupado	01/23/45/67	0.8238	0.2746	0.6089

Táboa 4.7: Comparativa en test entre a *baseline* ordinal e a reformulación de IVR en catro clases agrupadas.

Como se observa na Táboa 4.7, a mellora aparente do erro bruto de IVR non pode interpretarse como unha mellora real, xa que se produce nunha escala máis curta. Ao normalizar polo rango ordinal, a configuración agrupada queda lixeiramente por detrás da *baseline*. A comparación máis directa confirma a mesma tendencia: ao colapsar as predicións da *baseline* ao esquema 01/23/45/67, esta segue acadando un menor erro agrupado (0.8009 fronte a 0.8238) e unha maior exactitude exacta (0.6533 fronte a 0.6089).

A matriz de confusión, mostrada na Figura 4.11, confirmou ademais que a agrupación modificaba a distribución dos erros pero non eliminaba o problema principal. En particular, reduciuse o número de casos do grupo 01 preditos como 67, que pasaron de 90 a 44, pero empeorou o erro inverso: as mostras reais 67 preditas como 01 aumentaron de 42 a 81.

Polo tanto, a agrupación de IVR non resolveu de forma robusta a confusión entre extremos da escala, e decidiuse continuar explorando outras formas de incorporar a lóxica experta do problema.

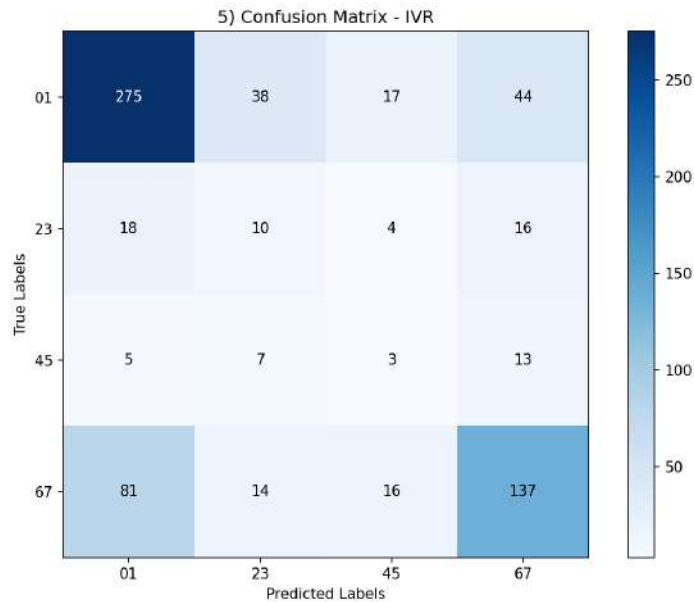


Figura 4.11: Matriz de confusión de IVR para a execución con catro clases agrupadas.

Aplicación da lóxica experta

Chegados a este punto do desenvolvemento, decidiuse aplicar a lóxica experta de forma directa na inferencia. Ata agora, estiveráanse tratando HPI e IVR como dúas tarefas de clasificación ordinal. Para HPI esta aproximación resultaba acorde coa lóxica experta, pero no caso de IVR mesturaba dous significados distintos. Como vimos, cando $HPI=0$, $HPI=5$ ou $HPI=6$, IVR debe tomar o valor 0 porque non aplica. Nos casos nos que si aplica, IVR xa non representa cantidade de dano, senón a posición da mostra na comparación entre marcas atribuídas a vertebrados e marcas atribuídas a invertebrados.

Esta diferenza semántica levou a separar dous problemas que ata entón se estaban mesturando nunha única saída, decidir se IVR debe aplicarse e estimar a nota correspondente cando si aplica. Nesta primeira aproximación non se engadiu unha cabeza independente para decidir a aplicabilidade, senón que se aproveitou directamente a predición de HPI. Só cando HPI está entre 1 e 4 ten sentido predicir unha nota de IVR. Deste xeito, a inferencia pasou a incorporar unha restrición biolóxica que o modelo ordinal anterior tiña que aprender de maneira implícita.

O cambio interno da CNN foi especialmente relevante. Mantívose o mesmo *backbone*

ConvNeXt_Tiny e a mesma cabeza ordinal para HPI ca nas execucións anteriores: para unha escala de sete clases, a rede produce $K-1$ saídas asociadas aos umbrais ordinais, adestradas con *BCEWithLogits*. A predición final de HPI obtense contando cantos destes limiares se superan, igual ca na formulación tipo CORAL empregada previamente. Polo tanto, a parte de HPI non se modificou.

A diferenza apareceu na saída de IVR. En lugar de predicir oito clases ordinais 0 . . 7, a rede pasou a producir un único valor continuo entre 0 e 1, obtido mediante unha sigmoide. Este valor representa o eixo entre predominio de marcas atribuídas a vertebrados e predominio de marcas atribuídas a invertebrados, seguindo a escala definida na Táboa 3.4. Para converter as etiquetas expertas 1 . . 7 a este espazo continuo empregouse a proporción I/V recollida nesa táboa, normalizada ao intervalo 0 . . 1. A perda de IVR calculouse mediante *MSE*, pero só nas mostras nas que a nota tiña sentido biolóxico, é dicir, cando o HPI real pertencía ao intervalo 1 . . 4, segundo o criterio descrito na Táboa 3.3.

Durante a inferencia, primeiro decodifícase HPI. Se a predición cae en 0, 5 ou 6, a saída final de IVR fórzase a 0. Se, pola contra, o HPI predito cae en 1 . . 4, úsase o valor continuo de IVR e asígnase á clase experta 1 . . 7 cuxa nota estea máis próxima. Este comportamento fai que a rede xa non interprete $IVR=0$ como un nivel máis da escala, senón como unha consecuencia da aplicabilidade deducida a partir de HPI.

Este cambio obrigou tamén a reformular a avaliación. A métrica antiga de IVR sobre todas as mostras mantívose como referencia histórica, pero deixou de ser a lectura principal, xa que mestura erros de aplicabilidade con erros na nota. Por este motivo refactorizouse o *script* de test para incorporar as métricas descritas na Sección 4.4.2: avaliación binaria da aplicabilidade de IVR, calidade da nota só cando IVR debía aplicarse e o modelo tamén a predixo, e consistencia interna entre HPI e IVR. Esta separación resulta clave para interpretar os resultados seguintes, porque un erro en IVR pode estar causado pola propia saída de IVR, pero tamén pode ser unha consecuencia arrastrada dunha predición incorrecta de HPI.

Os resultados desta primeira variante, resumidos na Táboa 4.8, amosaron unha lectura distinta segundo se empregasen as métricas antigas ou as novas. Nas métricas orixinais, a *baseline* ordinal seguía obtendo mellores valores de *mae_hpi*, *mae_ivr* e *mae_mean*. Con todo, esta lectura deixara de ser suficiente: o *mae_ivr* calculado sobre todas as mostras mestura nun único número erros de aplicabilidade e erros na nota, polo que xa non reflectía ben a realidade de IVR.

Ao aplicar a avaliación reformulada, a nova aproximación mostrou unha vantaxe máis relevante para o problema: a súa saída era coherente coa regra experta. A consistencia interna entre HPI e IVR pasou a ser perfecta, precisamente porque a regra se forza respecto do HPI predito: se o modelo predí un HPI no que IVR non debería aplicar, a saída de IVR queda a 0; se predí un HPI entre 1 e 4, permítese unha nota positiva. Esta consistencia, porén, non

implica que a aplicabilidade sexa sempre correcta respecto da etiqueta real. Cando a predición de HPI é incorrecta, o erro pode arrastrarse directamente á decisión de aplicabilidade de IVR.

Métrica	Baseline ordinal	Nova formulación	Lectura
mae_ivr orixinal	1.8250	1.9770	Empeora, pero mestura aplicabilidade e nota de IVR.
Consistencia HPI-IVR	0.9168	1.0000	A nova saída respecta sempre a restrición experta.
Precisión da aplicabilidade	0.7853	0.8241	Redúcense predicións positivas incorrectas de IVR.
Falsos positivos de aplicabilidade	114	76	Baixa o número de casos nos que se dá nota sen deber facelo.
Recall da aplicabilidade	0.9392	0.8018	Custo da formulación: deixa máis casos válidos sen nota.
mae condicional de IVR	1.4293	1.4213	Mantense practicamente igual no subconxunto comparable.
rmse condicional de IVR	2.4514	2.0933	Redúcese parte dos erros máis afastados cando se predi nota.

Táboa 4.8: Comparativa entre a *baseline* ordinal e a primeira variante con IVR continuo condicionado pola predición de HPI segundo a avaliación reformulada.

Ademais da comparación directa, a Táboa 4.9 recolle as métricas principais que pasaron a empregarse para analizar esta nova formulación. Esta separación permite ver que o problema xa non se interpreta como unha única escala ordinal de IVR, senón como dúas decisións encadeadas: primeiro determinar se IVR aplica e despois avaliar a calidade da nota só nos casos nos que esa nota ten sentido.

Por este motivo, aínda que as métricas orixinais desen un resultado peor, esta formulación pasou a considerarse a nova *baseline* para as seguintes probas. A decisión non se baseou nunha mellora directa do mae_ivr antigo, senón en que esa métrica xa non representaba axeitadamente o problema: penalizaba de forma conxunta a aplicabilidade e a nota, cando a guía experta as trata como decisións diferentes. A nova formulación aseméllase máis á solución experta, reduce os falsos positivos de aplicabilidade, fai desaparecer as contradicións entre HPI e IVR e permite avaliar a nota de IVR só no subconxunto no que realmente ten sentido.

4.4.5 Versión Final

A versión final mantivo a decisión tomada no apartado anterior, polo que a aplicabilidade final de IVR debía seguir a regra experta derivada de HPI, e non unha cabeza independente. Con todo, incorporouse un cambio importante na arquitectura para que a estimación da nota

Bloque de avaliación	Métrica	Valor	Interpretación
HPI ordinal	<i>mae_hpi</i>	0.5940	Erro medio da tarefa ordinal de HPI.
Aplicabilidade de IVR	<i>Accuracy</i>	0.7647	Exactitude da decisión IVR=0 fronte a IVR>0.
Aplicabilidade de IVR	Precisión	0.8241	Fiabilidade cando o modelo decide dar unha nota de IVR.
Aplicabilidade de IVR	<i>Recall</i>	0.8018	Proporción de casos reais aplicables que reciben nota.
Aplicabilidade de IVR	<i>F1-score</i>	0.8128	Equilibrio entre precisión e <i>recall</i> .
Nota condicional de IVR	Mostras avaliadas	356	Casos nos que IVR debía aplicarse e o modelo tamén deu nota.
Nota condicional de IVR	<i>mae</i>	1.4213	Erro medio da nota só no sub-conxunto comparable.
Nota condicional de IVR	<i>rmse</i>	2.0933	Penalización dos erros grandes na nota condicional.
Nota condicional de IVR	<i>Within 2</i>	0.7865	Proporción de notas cun erro máximo de dous niveis.
Consistencia interna	HPI-IVR	1.0000	Ausencia de contradicións entre HPI predito e IVR final.

Táboa 4.9: Métricas principais da avaliación reformulada para a variante con IVR continuo condicionado por HPI.

de IVR non se fixese só a partir das características visuais extraídas polo *backbone*. A cabeza de IVR pasou a recibir tamén a saída ordinal de HPI como entrada, de modo que a rede puidese usar explicitamente a información que ela mesma estaba inferindo sobre o tipo de herbivoría.

A arquitectura final emprega de novo *ConvNeXt_Tiny* preadestrado como extractor común de características. Sobre ese vector de características aplícase primeiro a cabeza de HPI, que se mantén igual ca nas variantes anteriores: unha capa lineal con $K-1$ logits, onde $K=7$, adestrada mediante entropía cruzada binaria sobre os umbrais ordinais. A predición de HPI obtense aplicando unha sigmoide a estes logits e contando cantos umbrais superan 0.5.

A diferenza está na cabeza de IVR. Xa non existe unha cabeza binaria de aplicabilidade. Primeiro calcúlanse as probabilidades ordinais de HPI; despois concaténanse co vector de características do *backbone*. Este vector combinado introdúcese nun pequeno adaptador formado por unha capa lineal, unha activación *ReLU* e *dropout* de 0.1. Finalmente, unha única saída produce o score continuo de IVR, que se transforma cunha sigmoide ao intervalo 0. . 1. Polo tanto, a rede aprende a nota de IVR condicionada pola información ordinal de HPI, pero a decisión de se IVR aplica segue resolvéndose mediante unha regra fixa na inferencia.

Durante o adestramento, HPI optimízase coa mesma perda ordinal tipo CORAL-like usada na *baseline*. Para IVR, as etiquetas expertas positivas transfórmanse ao mesmo espazo continuo empregado nas probas anteriores, tomando como referencia a proporción invertido/vertebrado da Táboa 3.4 normalizada ao intervalo 0. . 1. A perda de IVR é un *MSE* aplicado só ás mostras con $IVR > 0$, é dicir, aquelas nas que a nota existe. A perda total mantén os pesos 0.3/0.7 entre HPI e IVR. O adestramento realizouse con imaxes redimensionadas a 224 píxeles, *batch size* 8, *AdamW*, taxa de aprendizaxe 0.0001, *weight decay* 0.0001, *mixed precision* e *early stopping* con paciencia 8.

Na inferencia, o proceso completo queda definido do seguinte xeito:

1. A imaxe normalizada entra no *backbone ConvNeXt_Tiny*, que produce o vector de características común.
2. A cabeza ordinal de HPI produce seis logits. Tras aplicar a sigmoide, cóntanse os límites superiores a 0.5 para obter a clase final de HPI.
3. As probabilidades ordinais de HPI concaténanse co vector de características e aliméntanse á cabeza continua de IVR.
4. A saída de IVR transfórmanse nun score entre 0 e 1. Se o HPI predito pertence a 1. . 4, este score asígnase á clase 1. . 7 coa ancoraxe máis próxima.
5. Se o HPI predito é 0, 5 ou 6, a saída final de IVR fórzase a 0, independentemente do score continuo producido pola rede.

Esta inferencia é a diferenza central da solución final: IVR aprende unha nota continua apoiándose na información de HPI, pero a decisión final de aplicabilidade queda aliñada coa guía experta. Así evítase que a rede dea unha nota positiva de IVR cando o seu propio HPI indica que a variable non debería aplicarse.

Métrica	Baseline condicionada	Versión final
mae_hpi	0.5940	0.5768
Exactitude de HPI	0.5925	0.5940
mae_ivr histórico	1.9770	1.9139
Accuracy aplicabilidade	0.7647	0.7747
Precisión aplicabilidade	0.8241	0.8225
Recall aplicabilidade	0.8018	0.8243
Falsos negativos de aplicabilidade	88	78
mae condicional de IVR	1.4213	1.3907
rmse condicional de IVR	2.0933	2.1507
Exact match condicional	0.3455	0.3798
Within 1 condicional	0.6489	0.7022
Consistencia HPI-IVR	1.0000	1.0000

Táboa 4.10: Comparativa en test entre a *baseline* condicionada e a versión final.

Como se observa na Táboa 4.10, a versión final mantén a consistencia perfecta entre HPI e IVR, pero mellora varios puntos relevantes fronte á *baseline* condicionada. O *recall* de aplicabilidade sobe de 0.8018 a 0.8243, polo que se reducen os casos nos que a mostra debía ter nota de IVR pero o modelo a deixaba a 0. Ademais, no subconxunto no que IVR realmente aplica e o modelo tamén predí unha nota, o mae condicional baixa de 1.4213 a 1.3907 e o *within 1* sobe de 0.6489 a 0.7022. A única lectura menos favorable é que o rmse condicional non mellora respecto á *baseline* anterior, pero o conxunto das métricas indica unha solución máis equilibrada: mantén a lóxica experta estrita, aproveita mellor a información de HPI e mellora a calidade media da nota de IVR.

4.5 Integración e funcionamento da pipeline

Unha vez seleccionadas as versións finais do detector YOLO e da CNN, desenvolveuse un último *script*, denominado `script_inferencia.py`, co obxectivo de integrar todas as etapas anteriores nun único fluxo de traballo executable. A finalidade deste programa era aplicar a solución completa sobre un directorio de imaxes novas e producir unha saída directamente revisable polos expertos. Este *script* sería o que se usa polos expertos, polo que debería ser sinxelo.

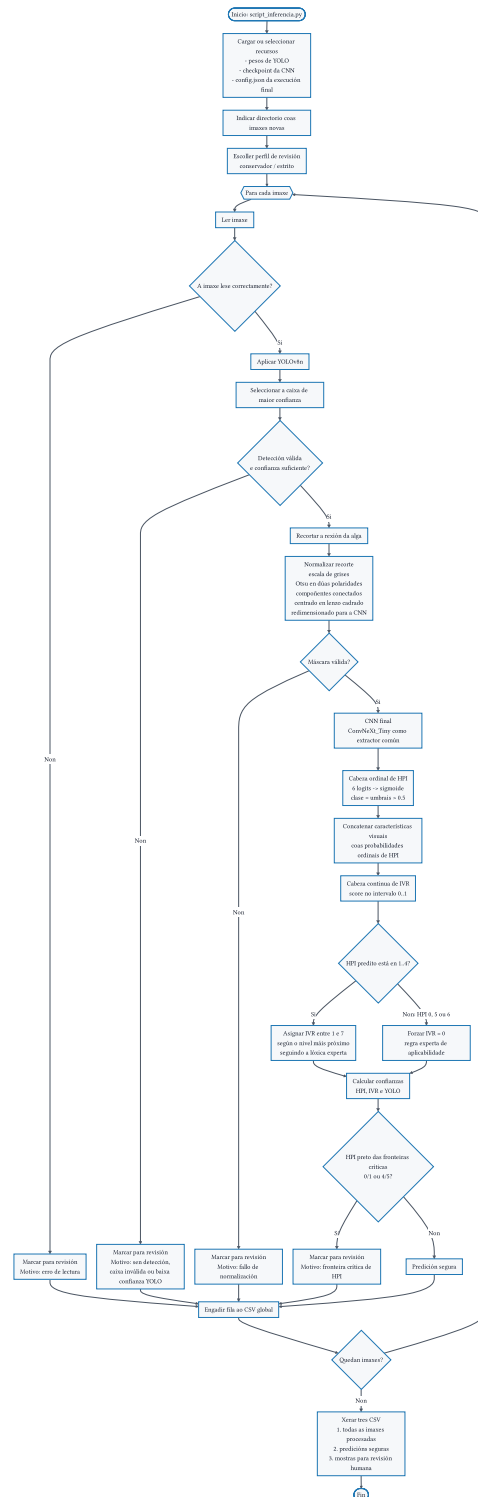
O *script* parte de tres elementos principais: o peso adestrado de YOLO, o peso adestrado da CNN e o ficheiro `config.json` asociado á execución final da rede. Este último ficheiro é necesario porque o *checkpoint* da CNN non contén por si só toda a información precisa para reconstruír a arquitectura. Por este motivo, ao comezo da execución compróbase a presenza destes ficheiros e, se hai máis dun candidato, permítese seleccionar interactivamente cal debe empregarse.

Despois desta comprobación inicial, o programa solicita a ruta do directorio coas imaxes que se queren analizar. Tamén permite escoller de forma interactiva o criterio de revisión manual entre `solo_dudoso` e `conservador`. Esta decisión introduciuse para que os expertos poidan distinguir entre as inferencias seguras e as que requiren revisión humana segundo a confianza da CNN. Os casos que se revisan son aqueles nos que HPI está preto das fronteiras 0/1 ou 4/5, dado que estas fronteiras son as que crean os erros de aplicabilidade que se pretenden reducir. A Táboa 4.11 resume os umbrais empregados en cada modo.

Modo	Marxe HPI	Rango de revisión	Interpretación
<code>solo_dudoso</code>	0.05	0.45–0.55	Marca unicamente os casos moi próximos ao cambio de decisión.
<code>conservador</code>	0.15	0.35–0.65	Marca máis casos preto das fronteiras 0/1 e 4/5.

Táboa 4.11: Criterios de revisión manual segundo a distancia ás fronteiras críticas de HPI.

De forma esquemática, a Figura 4.12 resume o funcionamento completo da *pipeline* de inferencia, dende a carga dos modelos ata a creación dos ficheiros CSV finais. Co fin de facilitar a lectura do fluxo, o diagrama represéntase nunha páxina independente.

Figura 4.12: Diagrama de fluxo da *pipeline* de inferencia desenvolvida.

A execución da pipeline realízase en tres fases. En primeiro lugar, aplícase YOLO a cada imaxe e selecciónase a caixa de maior confianza. Se non se detecta ningunha alga, se a caixa é inválida ou se a imaxe non pode lerse correctamente, a mostra márcase directamente como pendente de revisión. No caso contrario, recórtase a rexión detectada e gárdase temporalmente para a seguinte etapa.

En segundo lugar, cada recorte normalízase reutilizando a mesma lóxica empregada durante o desenvolvemento do conxunto de adestramento. O recorte convértese a escala de grises, aplícase Otsu nas dúas polaridades posibles e consérvanse os compoñentes conectados relevantes, priorizando a opción que evita ocupar excesivamente os bordes da imaxe. A máscara resultante céntrase nun lenzo cadrado e redimensiónase ao tamaño común necesario para a CNN. Se a segmentación non produce unha máscara válida, a mostra tamén se deriva a revisión manual.

Na terceira fase, as imaxes normalizadas introdúcense na CNN final. A cabeza ordinal de HPI produce seis logits, que se transforman mediante sigmoide e se decodifican contando os limiares superiores a 0.5. A saída continua de IVR decodifícase mediante as ancoraxes definidas no `config.json`, pero só se utiliza cando o HPI predito está no rango 1..4. Deste xeito, a ferramenta mantén exactamente a regra experta descrita na versión final do modelo e evita contradicións entre ambas variables.

Como resultado, o programa xera tres ficheiros CSV. O primeiro contén todas as imaxes procesadas; o segundo inclúe só aquelas predicións consideradas seguras; e o terceiro recolle as mostras que requiren revisión humana. Co fin de que a saída sexa útil para os expertos e non dependa de detalles internos da implementación, estes ficheiros conteñen unicamente as columnas necesarias para a revisión: identificador da imaxe, predición de HPI, predición de IVR, confianza de HPI, confianza de IVR, confianza de YOLO, indicador de revisión e motivo polo que a mostra foi marcada. Entre estes motivos inclúense fallos de detección, baixa confianza do detector, problemas de normalización ou proximidade ás fronteiras críticas de HPI.

Os motivos polos que a *pipeline* pode marcar unha mostra para revisión manual son variados. Como se indicou ao comezo desta sección, o obxectivo principal é sinalar aqueles casos nos que a CNN pode ser máis propensa a cometer erros, especialmente cando a predición de HPI se atopa preto das fronteiras críticas que condicionan a aplicabilidade de IVR.

Ademais, o sistema tamén marca para revisión as imaxes nas que se produce algún erro durante a execución da *pipeline*. Nestes casos, o motivo non está relacionado coa incerteza da rede, senón cun problema detectado nalgunha das etapas anteriores, como a lectura da imaxe, a detección da alga ou a normalización do recorte. Deste xeito, os expertos poden identificar por que unha mostra non foi procesada correctamente e realizar a avaliación do dano de maneira manual. A explicación de todos os motivos polos que se pode activar o *flag*

de revisión atópanse na Figura 4.12

Motivo de revisión	Interpretación
hpi_boundary_0_1	A predición de HPI está preto da fronteira entre ausencia de dano e dano leve.
hpi_boundary_4_5	A predición de HPI está preto da fronteira entre dano na lámina e perda da lámina.
Sen detección	YOLO non detectou ningunha alga na imaxe.
Caixa inválida	A rexión detectada non permite obter un recorte válido.
Baixa confianza YOLO	A detección existe, pero a confianza do detector é insuficiente.
Erro de normalización	Non foi posible xerar unha máscara válida ou unha entrada adecuada para a CNN.
Erro de lectura	A imaxe non puido ser aberta ou procesada correctamente.

Táboa 4.12: Motivos polos que unha mostra pode ser marcada para revisión manual pola *pipeline*.

Na Táboa 4.13 móstrase un exemplo simplificado do formato destes CSV. As dúas primeiras mostran non precisan revisión, mentres que as dúas últimas se marcan por atoparse preto das fronteiras de decisión que afectan á aplicabilidade de IVR.

Imaxe	HPI	IVR	Conf. HPI	Conf. IVR	Conf. YOLO	Rev.	Motivo
peer_078	3	5	0,1452	0,0808	0,8067	0	
peer_079	5	0	0,8004	0,3641	0,7355	0	
peer_080	4	1	0,6117	0,0033	0,5338	1	hpi_boundary_4_5
peer_081	0	0	0,6384	0,1388	0,5783	1	hpi_boundary_0_1

Táboa 4.13: Exemplo do formato dos CSV xerados pola pipeline de inferencia.

Deste xeito, `script_inferencia.py` actúa como a peza de integración final do sistema porque encadea detección, recorte, normalización e inferencia, aplica as regras expertas necesarias e produce unha saída sinxela de interpretar. Ao mesmo tempo, conserva un comportamento trazable, xa que cada mostra marcada para revisión indica explicitamente a causa pola que non foi considerada segura.

4.6 Resultados xerais da pipeline final

Unha vez integrada a solución completa e descrito o seu funcionamento na sección anterior, apoiándose no diagrama da Figura 4.12, avalíouse o comportamento da *pipeline* final como sistema extremo a extremo. Esta avaliación considera de forma conxunta a detección con YOLO, a normalización do recorte, a inferencia da CNN e a aplicación das regras expertas de revisión. Nesta sección preséntanse os resultados globais obtidos e algúns exemplos cualitativos representativos da saída final do sistema.

4.6.1 Resumo cuantitativo dos resultados

Nesta subsección recóllense de forma conxunta as métricas principais obtidas pola versión final da *pipeline*. A diferenza das avaliacións anteriores, aquí non se analiza cada compoñente de forma illada, senón o comportamento global do sistema xa integrado.

Os resultados cuantitativos dos compoñentes principais xa foron presentados nas seccións anteriores. O detector YOLOv8n acadou un comportamento estable na detección da alga, como se recolle na Táboa 4.1, mentres que a versión final da CNN obtivo as métricas resumidas na Táboa 4.10. Nesta sección non se repiten esas avaliacións por separado, senón que se interpretan de forma conxunta para valorar o comportamento da *pipeline* completa.

Na execución final sobre o subconxunto de 200 imaxes empregado posteriormente na validación con expertos da Sección 5.1, analizouse tamén o comportamento operativo da *pipeline*. En particular, contabilizouse cantas mostrase eran devoltas como predicións seguras e cantas eran marcadas para revisión manual segundo os dous perfís descritos na Táboa 4.11.

Perfil de revisión	Imaxes totais	Marcadas para revisión	Predicións seguras
solo_dudoso	200	3	197
conservador	200	15	185

Táboa 4.14: Resumo operativo da execución final da *pipeline* sobre o subconxunto de validación con expertos.

Na execución co perfil *conservador*, 13 das 15 imaxes marcadas para revisión tiñan como motivo a proximidade da predición de HPI ás fronteiras críticas 0/1 ou 4/5. As outras dúas mostrase foron marcadas por baixa confianza de YOLO, aínda que unha revisión manual posterior confirmou que a alga fora segmentada correctamente nestes casos.

Pola súa banda, na execución co perfil *solo_dudoso*, as 3 imaxes marcadas para revisión correspondían a casos situados preto da fronteira 0/1. Este resultado é coherente coa definición máis restritiva deste perfil, que só sinala mostrase moi próximas ao cambio de decisión.

4.6.2 Resumo cualitativo dos resultados

Para complementar a análise cuantitativa, nesta subsección preséntanse exemplos reais da saída da *pipeline*. O obxectivo é ilustrar tanto o comportamento do sistema en casos problemáticos como a utilidade dos mecanismos de revisión manual introducidos na versión final.

Exemplo de fallo

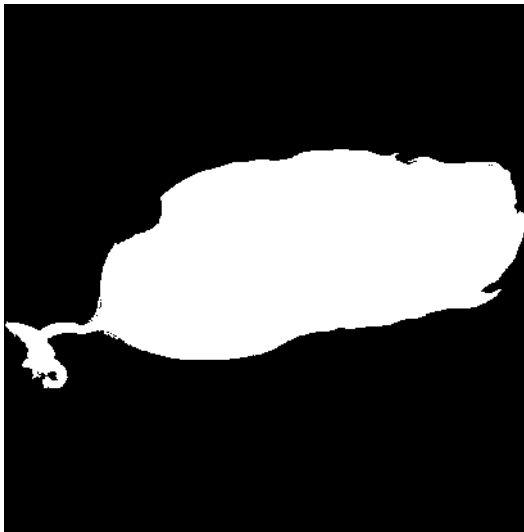
Neste primeiro exemplo, os dous modos de execución (`conservador` e `solo_dudoso`) marcaron a mostra para revisión. Polo tanto, aínda que o sistema fallou na aplicabilidade de IVR ao predicir $HPI=0$ e, como consecuencia, $IVR=0$, a *pipeline* non considerou a saída como segura e solicitaría unha revisión experta manual. Segundo a etiqueta orixinal do *dataset*, a mostra correspondía a $HPI=1$ e $IVR=7$.



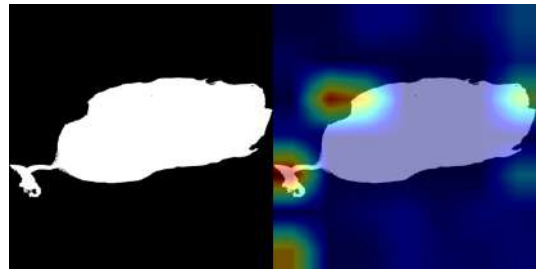
(a) Imaxe orixinal.



(b) Detección realizada por YOLO.



(c) Recorte normalizado introducido na CNN.



(d) *Heatmap* asociado á predición de HPI.

Figura 4.13: Exemplo dunha mostra marcada para revisión pola proximidade á fronteira $HPI=0/1$.

Como se observa na Figura 4.13 e na Táboa 4.15, a CNN inferiu $HPI=0$. Porén, a confianza asociada a HPI quedou nun valor próximo á fronteira de decisión $0/1$, polo que a mostra foi marcada correctamente para revisión. O *heatmap* indica ademais que a rede non concentra a súa atención unicamente na mordedura, o que puido contribuir a desprazar a decisión cara

Imaxe	HPI	IVR	Conf. HPI	Conf. IVR	Conf. YOLO	Rev.	Motivo
peer_056	0	0	0,5247	0,0248	0,9613	1	hpi_boundary_0_1

Táboa 4.15: Fila xerada pola *pipeline* para o exemplo de fallo.

á ausencia de dano. Este erro en HPI arrastra directamente a saída de IVR, xa que a lóxica experta da *pipeline* forza IVR=0 cando a predición de HPI indica ausencia de dano.

Exemplo de acerto

Como contrapunto ao caso anterior, a Figura 4.14 mostra unha mostra na que a *pipeline* predixo correctamente tanto HPI como IVR. Segundo a etiqueta orixinal do *dataset*, esta imaxe correspondía a HPI=4 e IVR=6, valores que foron reproducidos pola versión final do sistema.

Imaxe	HPI	IVR	Conf. HPI	Conf. IVR	Conf. YOLO	Rev.	Motivo
peer_022	4	6	0,6750	0,0389	0,4654	0	

Táboa 4.16: Fila xerada pola *pipeline* para o exemplo de acerto.

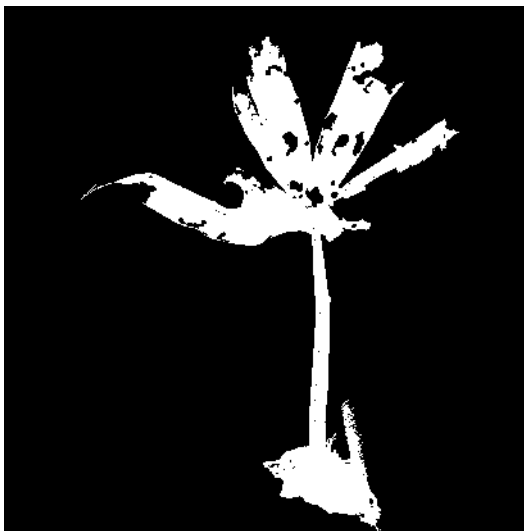
Como se recolle na Táboa 4.16, a mostra non foi marcada para revisión nin no perfil *solo_dudoso* nin no perfil *conservador*. A confianza de YOLO non é tan elevada como noutros exemplos, pero supera o umbral empregado pola *pipeline* e a detección da alga é suficiente para xerar un recorte válido. A partir dese recorte, a CNN asigna correctamente HPI=4 e IVR=6. O *heatmap* da Figura 4.14 amosa que a rede concentra parte da súa atención nas zonas da lámina nas que aparecen marcas de herbivoría, o que resulta coherente coa predición obtida.



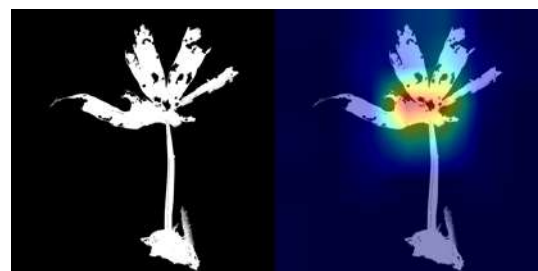
(a) Imaxe orixinal.



(b) Detección realizada por YOLO.



(c) Recorte normalizado introducido na CNN.



(d) Heatmap asociado á predición de IVR.

Figura 4.14: Exemplo dunha mostra clasificada correctamente pola *pipeline*.

Conclusións

NESTE derradeiro capítulo recóllese a validación realizada cos expertos a partir dunha reunión e dunha proba práctica do sistema. Ademais, resúmense as principais conclusións do traballo, analízanse as limitacións detectadas e propóñense posibles liñas futuras de mellora.

5.1 Validación con expertos

Nos últimos días de desenvolvemento do sistema, cando xa se dispoñía da versión final da CNN, organizouse unha reunión con expertos da Facultade de Ciencias. Máis concretamente, participaron membros do grupo BIOCOST (Bioloxía Costeira), que cederan o material empregado durante este traballo. A reunión contou coa participación de Sara Barrientos de la Llana, investigadora postdoutoral, e Nerea Alvite Bergara, investigadora predoutoral, ambas pertencentes ao grupo mencionado.

5.1.1 Resumo da reunión

A reunión comezou cunha presentación do sistema desenvolvido, na que se explicaron as métricas empregadas na avaliación da CNN e se mostraron os resultados obtidos coa versión final. A recepción foi positiva unha vez explicado o efecto de arrastre que un erro en HPI pode producir sobre a aplicabilidade de IVR, así como o significado das novas métricas condicionais, o equipo experto indicou que os resultados eran coherentes co comportamento esperado.

Aínda así, co obxectivo de realizar unha comprobación máis obxectiva, propúxose unha proba práctica: as dúas investigadoras avaliarían manualmente un subconxunto de 200 imaxes extraídas do *dataset*. Este subconxunto non se xerou mediante unha mostraxe completamente aleatoria, senón mediante unha selección estratificada para que estivese representada toda a escala de HPI e, dentro dos casos nos que IVR aplica, tamén a diversidade das notas de IVR.

Para iso, fixouse un mínimo aproximado do 10% das imaxes para cada valor de HPI e, nos valores 1 . . 4, procurouse repartir tamén as mostras entre as clases positivas de IVR. Nos casos HPI=0, HPI=5 e HPI=6, a etiqueta IVR mantívose en 0, seguindo a lóxica experta descrita anteriormente.

HPI	Imaxes	Distribución de IVR							
		0	1	2	3	4	5	6	7
0	34	34	0	0	0	0	0	0	0
1	47	0	7	6	5	5	5	7	12
2	27	0	5	4	3	3	3	4	5
3	23	0	4	3	3	3	3	3	4
4	22	0	5	3	3	3	3	2	3
5	26	26	0	0	0	0	0	0	0
6	21	21	0	0	0	0	0	0	0

Táboa 5.1: Distribución do subconxunto de 200 imaxes empregado para a avaliación manual por parte dos expertos.

Durante a xeración deste subconxunto só se detectou unha limitación: para a combinación HPI=4 , IVR=6 había dúas imaxes dispoñibles, aínda que o obxectivo mínimo eran tres. Neste caso empregouse a mellor aproximación posible, sen alterar o tamaño total do subconxunto. Ademais, exportáronse dous ficheiros separados: un modelo de revisión para os expertos, sen información das etiquetas orixinais, e un CSV privado coas respostas para poder comparar posteriormente as avaliacións.

Con esta avaliación manual, buscábase comprobar ata que punto o sistema desenvolvido se podía comparar coa avaliación experta. Ademais, a proba permitía analizar se, mesmo entre persoas expertas, a variabilidade da nota podía ser elevada en certos casos pola subxectividade da detección visual do dano ou da atribución da depredación.

5.1.2 Resultados da proba

Unha vez se recibiu a resposta das investigadoras, realizouse a inferencia das mesmas 200 imaxes por parte do noso sistema. Esta execución fíxose cos dous perfís de revisión manual descritos no capítulo anterior: `solo_dudoso` e `conservador`. Cómpre sinalar que estes dous perfís non modifican a predición da CNN, senón unicamente o criterio empregado para marcar unha mostra como potencialmente revisable. De feito, nas 200 imaxes avaliadas as predicións de HPI e IVR foron idénticas nos dous perfís; a única diferenza foi o número de imaxes sinaladas para revisión, con 3 casos no perfil `solo_dudoso` e 15 no perfil `conservador`.

Variabilidade entre expertas

En primeiro lugar analizouse o grao de acordo entre as dúas investigadoras e a súa relación coa etiqueta orixinal do *dataset*. Esta análise é importante porque a etiqueta orixinal non debe interpretarse como unha medida física exacta, senón como unha avaliación experta previa. Polo tanto, antes de valorar o sistema é necesario coñecer que marxe de variabilidade aparece cando diferentes persoas expertas revisan as mesmas imaxes.

Para HPI empregáronse métricas ordinais directas: *MAE*, exactitude e acerto dentro de ± 1 punto. Para IVR, mantendo a formulación descrita na Sección 4.4.2, separouse a aplicabilidade (IVR=0 fronte a IVR=1 . . . 7) da calidade da nota cando as dúas avaliacións comparadas consideran que IVR aplica.

Comparación	HPI MAE	HPI ± 1	IVR aplic.	F1	IVR cond.	MAE	IVR cond.	± 1
Sara – Nerea	0.190	0.995	0.965		0.500		0.912	
Orixinal – Sara	0.465	0.945	0.918		1.593		0.619	
Orixinal – Nerea	0.525	0.945	0.879		1.888		0.560	

Táboa 5.2: Variabilidade entre as dúas investigadoras e respecto da etiqueta orixinal. A métrica de IVR condicional calcúlase só cando as dúas avaliacións comparadas asignan unha nota positiva de IVR.

Os resultados da Táboa 5.2 mostran un acordo moi alto entre Sara e Nerea. En HPI, o erro medio entre ambas foi de só 0.190 puntos e practicamente todas as mostras quedaron dentro de ± 1 punto. En IVR tamén se observou un acordo elevado na aplicabilidade, cun F1 de 0.965, e unha diferenza media de 0.500 puntos nos casos nos que ambas asignaron unha nota positiva.

Porén, a comparación de cada investigadora coa etiqueta orixinal presenta unha variabilidade maior. En HPI, o erro medio fronte ao dato orixinal sitúase entre 0.465 e 0.525 puntos. En IVR, a diferenza é máis acusada: o *MAE* condicional fronte ao orixinal sobe ata 1.593 no caso de Sara e 1.888 no caso de Nerea.

Pódese interpretar estes datos como que a avaliación experta actual, aínda que teña un erro maior en IVR que en HPI, é máis consistente entre expertas que o histórico que se usou para adestrar o sistema. Isto podería explicar inconsistencias posteriores entre a inferencia do sistema e as expertas.

Sistema fronte ao dato orixinal

A segunda análise comparou directamente a predición do sistema coa etiqueta orixinal do subconxunto de 200 imaxes. Esta comparación permite comprobar se o comportamento observado na proba práctica é coherente cos resultados obtidos previamente no conxunto de

test.

Métrica	Test final	Proba expertos
HPI MAE	0.5768	0.3600
Recall aplicabilidade IVR	0.8243	0.8824
IVR MAE condicional	1.3907	1.0286
Within 1 condicional de IVR	0.7022	0.7143

Táboa 5.3: Comparación entre as métricas obtidas no test final e as obtidas polo sistema fronte á etiqueta orixinal no subconxunto revisado polos expertos.

Como se observa na Táboa 5.3, o sistema obtivo na proba de expertos resultados iguais ou mellores ca os rexistrados no test final. O MAE de HPI baixou de 0.5768 a 0.3600 e a exactitude subiu ata 0.7400. En IVR, a mellora tamén aparece de forma consistente. A aplicabilidade acadou un *recall* de 0.8824 e a nota condicional presenta un MAE de 1.0286.

Esta lectura debe facerse con cautela, xa que o subconxunto de validación con expertos non foi extraído mediante mostraxe aleatoria simple, senón mediante unha selección estratificada para cubrir toda a escala de HPI e diferentes niveis positivos de IVR. Aínda así, o resultado é positivo porque, nun conxunto deseñado precisamente para conter diversidade de casos, o sistema mantén un comportamento coherente e mesmo lixeiramente mellor.

Sistema fronte ás investigadoras

A terceira comparación avaliou se o sistema se comporta como un avaliador adicional fronte ás dúas investigadoras. Para iso comparouse o resultado sistema-experta co resultado Sara-Nerea.

Comparación	HPI MAE	HPI ± 1	IVR aplic. F1	IVR cond. MAE	IVR cond. ± 1
Sara – Nerea	0.190	0.995	0.965	0.500	0.912
Sara – Sistema	0.635	0.910	0.872	2.027	0.545
Nerea – Sistema	0.615	0.910	0.860	2.263	0.456

Táboa 5.4: Comparación do sistema coas dúas investigadoras fronte á variabilidade observada entre elas.

A Táboa 5.4 mostra unha diferenza clara entre HPI e IVR. En HPI, o sistema queda máis afastado das investigadoras do que elas quedan entre si, pero segue mantendo unha proximidade razoable debido a que nos dous casos máis do 91% das mostras quedan dentro de ± 1 punto. Isto indica que, para a variable principal de intensidade de herbivoría, o sistema produce valoracións próximas ás humanas aínda que non alcanza o nivel de acordo entre expertas.

En IVR, pola contra, a diferenza sistema–experta é máis alta. A aplicabilidade segue presentando valores de F1 aceptables, entre 0.860 e 0.872, pero a nota condicional acada erros medios de 2.027 e 2.263 puntos, claramente superiores ao 0.500 observado entre Sara e Nerea. Este resultado matiza que o sistema pode aproximarse ben á etiqueta orixinal e manter unha estrutura coherente coas regras expertas, pero a súa saída de IVR non reproduce con tanta precisión o criterio compartido polas dúas investigadoras nesta revisión manual.

Unha interpretación plausible é que a etiqueta orixinal do *dataset* e as novas revisións non representan exactamente o mesmo punto de referencia. De feito, na Táboa 5.2 xa se observa que as investigadoras se afastan máis da etiqueta orixinal en IVR ca entre elas. Polo tanto, cando o sistema se compara coas investigadoras, parte do desacordo pode proceder de que foi adestrado para reproducir a etiqueta orixinal, non unha nova etiqueta consensuada xerada a posteriori.

Análise das mostras marcadas para revisión

Por último, avalíouse se os perfís de revisión manual sinalaban imaxes nas que realmente existía maior incerteza. Para isto analizouse a variabilidade entre a etiqueta orixinal, Sara e Nerea dentro dos subconxuntos marcados. Como se describiu na Táboa 4.11, o perfil `solo_dudoso` marca só os casos máis próximos ás fronteiras críticas de HPI, mentres que o perfil `conservador` emprega unha marxe máis ampla e, polo tanto, deriva máis mostras á revisión manual.

Subconxunto	n	HPI MAE pares	IVR MAE pares	IVR desac. aplic.
Todas as mostras	200	0.393	1.443	0.160
Marcadas para revisión en <code>solo_dudoso</code>	3	0.222	1.286	0.333
Seguras en <code>solo_dudoso</code>	197	0.396	1.445	0.157
Marcadas para revisión en <code>conservador</code>	15	0.578	2.163	0.400
Seguras en <code>conservador</code>	185	0.378	1.387	0.141

Táboa 5.5: Variabilidade entre etiqueta orixinal, Sara e Nerea nos subconxuntos sinalados polos perfís de revisión. O MAE por pares calcula a diferenza media entre todos os pares dispoñibles no subconxunto, mentres que o desacordo de aplicabilidade de IVR mide diferenzas entre $IVR=0$ e $IVR>0$.

Os resultados da Táboa 5.5 apoian a utilidade do perfil `conservador`. As 15 mostras que este perfil marca para revisión presentan máis variabilidade ca o conxunto completo: o HPI MAE por pares sobe de 0.393 a 0.578, o IVR MAE por pares aumenta de 1.443 a 2.163.

O perfil `solo_dudoso`, pola contra, é demasiado restritivo para extraer conclusións estatísticas robustas nesta proba, xa que só sinala 3 imaxes. Ademais, esas tres mostras non

presentan maior variabilidade ordinal en HPI nin en IVR ca o conxunto completo; o sinal que si aparece é un desacordo de aplicabilidade de IVR superior ao global. En conxunto, esta análise suxire que o mecanismo de revisión manual é útil especialmente na configuración conservadora, xa que tende a seleccionar imaxes nas que tamén existe maior variabilidade entre avaliacións humanas.

Conclusión da validación

Tomando a proba no seu conxunto, os resultados apoian parcialmente a hipótese de que o sistema pode actuar como unha ferramenta de apoio comparable a unha avaliación experta. Fronte á etiqueta orixinal, o sistema obtén resultados moi competitivos e mesmo mellores ca os observados no test final, mantendo ademais a consistencia estrutural entre HPI e IVR. Isto indica que a solución desenvolvida xeneraliza correctamente ao subconxunto revisado e reproduce de forma razoable o criterio co que foi adestrada.

Con todo, a comparación directa coas dúas investigadoras mostra que o sistema aínda non alcanza o nivel de acordo existente entre expertas, sobre todo na nota condicional de IVR. A lectura máis axeitada non é, polo tanto, que o sistema substitúa plenamente unha revisión experta, senón que pode automatizar unha parte importante do proceso e proporcionar puntuacións coherentes, reservando para revisión humana os casos máis incertos. A análise dos perfís de revisión reforza esta interpretación: o modo conservador marca un subconxunto pequeno de imaxes que efectivamente presenta maior variabilidade entre a etiqueta orixinal e as avaliacións das investigadoras.

5.2 Conclusiones do traballo

Nesta sección recóllense as conclusións xerais do traballo a partir dos obxectivos formulados ao comezo da memoria e da validación realizada co equipo experto.

Como se indicou na introdución, o traballo xurdiu a partir dunha necesidade do grupo BIOCOST da Facultade de Ciencias da UDC. Este equipo dispoñía dun historial amplo de algas xa analizadas e precisaba unha ferramenta capaz de axudar ás persoas expertas a estimar o nivel de dano e a posible orixe da depredación por herbivoría.

5.2.1 Cumprimento dos obxectivos

Coas probas descritas na Sección 5.1, pódese concluír que se acadaron os obxectivos principais do traballo. Analizouse e preparouse o conxunto de datos, deseñouse unha etapa de detección e normalización das imaxes, desenvolveuse unha CNN para estimar HPI e IVR, integrouse o fluxo completo nunha ferramenta executable e avaliouse o seu comportamento con datos reais e cunha revisión externa por parte de expertas.

Os resultados non demostran que o sistema poida substituír a avaliación experta, nin ese era o obxectivo do proxecto. O que si mostran é que pode automatizar unha parte relevante do proceso: proporciona estimacións coherentes nos casos máis claros e identifica mostras que convén revisar manualmente cando a decisión é máis incerta. Neses casos, a saída do sistema pode servir como valoración orientativa para apoiar o traballo experto.

Na reunión co equipo de BIOCOST, as investigadoras valoraron positivamente os resultados obtidos e consideraron coherente o aumento do erro en *IVR*, especialmente polo efecto de arrastre que pode producir unha predición incorrecta de *HPI*. Ademais, indicaron que a ferramenta podería poñerse en práctica no seu fluxo de traballo, o que reforza a utilidade aplicada da solución desenvolvida.

5.3 Limitacións do sistema

Malia os resultados obtidos, o sistema presenta varias limitacións que deben terse en conta para interpretar correctamente o seu alcance.

5.3.1 Erro en *IVR*

O principal problema da solución desenvolvida é o erro asociado á inferencia de *IVR*. Como se analizou ao longo da memoria, unha parte deste erro procede do arrastre de *HPI*. Se o sistema predí unha clase de *HPI* na que *IVR* non debería aplicarse, a saída final de *IVR* tamén queda afectada. Con todo, a validación coas investigadoras mostrou que o erro do sistema en *IVR* segue sendo claramente superior ao observado entre expertas, como se recolle na Táboa 5.4.

Esta dificultade pode deberse á propia natureza da variable, que require distinguir a procedencia das marcas de herbivoría e non só a intensidade do dano, pero tamén á variabilidade das etiquetas históricas empregadas para o adestramento.

5.3.2 Erro en *HPI*

Aínda que con menor severidade ca en *IVR*, o sistema tamén presenta máis variabilidade en *HPI* ca a observada entre as dúas expertas durante a revisión manual. Este comportamento pode estar relacionado coa variabilidade das propias etiquetas do *dataset*, xeradas ao longo dun período amplo e baixo criterios que foron refinándose co tempo.

5.3.3 Adestramento orientado a lóxica experta

Como se puido ver durante o desenvolvemento da CNN, a lóxica experta de aplicabilidade de *IVR* incorporouse nas últimas iteracións do traballo. Dado que o obxectivo era reproducir

o criterio empregado polos especialistas, tería sido preferible integrar esta lóxica desde as primeiras probas e definir desde o inicio métricas aliñadas coa interpretación biolóxica do problema.

En particular, a métrica global `mae_ivr` resultou útil para comparar experimentos iniciais, pero non representaba axeitadamente a realidade condicional de *IVR*, xa que mesturaba nun só valor os erros de aplicabilidade e os erros na nota cando a variable si tiña sentido.

5.3.4 Limitacións do conxunto de datos e do preprocesado

O traballo tamén estivo condicionado pola calidade e a heteroxeneidade do material inicial. As imaxes presentaban diferenzas de iluminación, fondo, orientación, escala e presenza de elementos auxiliares, o que obrigou a introducir unha etapa de detección con YOLO e unha normalización posterior. Aínda que esta estratexia funcionou de maneira suficiente para o sistema final, algunhas imaxes quedaron sen detección ou tiveron que descartarse tras a revisión manual dos recortes.

Ademais, a limpeza inicial aplicaba a comprobación de correspondencia nun único sentido. Esta eliminaba as imaxes sen referencia válida nas etiquetas, pero non retiraba automaticamente as filas do ficheiro tabular cuxa imaxe asociada non estivese dispoñible. Isto non tivo un impacto práctico nas execucións finais, porque as etapas posteriores filtraban as mostras dispoñibles, pero é un aspecto mellorable desde o punto de vista da consistencia do pipeline.

5.4 Liñas futuras

Finalmente, recóllense varias liñas de traballo que poderían mellorar a precisión do sistema e a súa utilidade no contexto real de uso.

5.4.1 Readestramento con datos novos

Como se observa na Táboa 5.2, as anotacións das dúas expertas presentan moita menos variabilidade entre si ca fronte ás etiquetas orixinais empregadas para adestrar o sistema. Isto suxire que o *dataset* histórico, construído ao longo de varios anos, podería incluír avaliacións realizadas baixo criterios non completamente homoxéneos ou anteriores ao grao de madurez actual da métrica experta.

Unha liña futura clara sería reavaliar, total ou parcialmente, o conxunto de imaxes con criterios actualizados e consensuados. Aínda que esta tarefa sería custosa, permitiría adestrar o modelo cun obxectivo máis próximo ao criterio experto actual e podería reducir especialmente o erro observado en *IVR*.

5.4.2 Estandarización de toma de fotografías

Outra mellora relevante sería estandarizar a toma de fotografías. Empregar unha distancia de captura, un fondo, unha iluminación e unha posición da mostra máis controladas reduciría a variabilidade allea ao fenómeno biolóxico estudado. Isto facilitaría a detección da alga por parte de YOLO e permitiría que a CNN se centrase mellor nos trazos relacionados co dano e coa súa procedencia, en lugar de aprender posibles patróns asociados ao ruído introducido polas condicións de captura.

5.4.3 Mellora da ferramenta de revisión

Unha continuación natural do sistema sería desenvolver unha interface máis completa para a revisión manual das mostras marcadas como incertas. A ferramenta actual xa xera ficheiros CSV diferenciando predicións seguras e mostras revisables, pero unha interface visual permitiría validar ou corrixir esas predicións de maneira máis cómoda e almacenar novas etiquetas revisadas para futuros readestramentos.

Apéndices

Repositorio do proxecto

O código fonte desenvolvido durante o proxecto, xunto cos scripts de preprocesado, adestramento, avaliación e inferencia, atópase dispoñible no seguinte repositorio:

<https://github.com/estev-o/TFG>

Glosario

- Backbone** En aprendizaxe profunda, o *backbone* é a parte dunha rede neuronal que se encarga de extraer características relevantes dos datos de entrada. Normalmente, é unha arquitectura preentrenada que se utiliza como base para tarefas específicas, como clasificación ou detección de obxectos.. [40](#), [41](#)
- CNN** Rede Neuronal Convolutacional. Arquitectura de aprendizaxe profunda especialmente eficaz para o procesado de imaxes e sinais mediante capas de convolución.. [2](#), [3](#), [15](#), [55](#), [74](#)
- CORAL** No marco de regresión ordinal para redes neuronais, CORAL é un método que garante monotonía no ranking, puntuación de confianza coherente e independencia da arquitectura. . [7](#)
- HPI** Índice de presión de depredación herbívora. É unha medida que avalía o dano por herbívoros que ten unha plante, no noso caso unha alga Laminaria. Un valor baixo (próximo a 0) indica pouco dano, e un valor alto (próximo a 6) indica danos maiores á alga). [v](#), [2](#), [3](#), [11–13](#), [15](#)
- IVR** Ratio de vertebrado vs invertebrado. Cuantifica, entre vertebrados e invertebrados, o número de mordidas. Sendo 0 non aplica, 1 sería 100% mordidas de peixes e 7 sería 100% mordidas de invertebrados, cos valores medios correspondentes.. [v](#), [2](#), [3](#), [11](#), [12](#), [14](#), [15](#)
- Kelp / Laminaria** *Kelp*: nome común de grandes algas laminarias en inglés. *Laminaria* é un xénero típico de algas en mares fríos e temperados.. [9](#)
- Softmax** Función de activación que transforma un vector de valores en probabilidades, asignando a cada valor una probabilidade entre 0 y 1, y asegurando que la suma de todas las probabilidades sea igual a 1. Es comúnmente utilizada en la capa de salida de modelos de clasificación multiclase.. [40](#)

YOLO É unha serie de sistemas de detección de obxectos en tempo real baseado en [CNNs](#). Esta tecnoloxía recibiu melloras ata converterse no *framework* máis popular de detección de obxectos.. [23](#)

Bibliografía

- [1] Z. Liu, H. Mao, C. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, “A convnet for the 2020s,” *CoRR*, vol. abs/2201.03545, 2022. [En línea]. Disponible en: <https://arxiv.org/abs/2201.03545>
- [2] J. Redmon, S. K. Divvala, R. B. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” *CoRR*, vol. abs/1506.02640, 2015. [En línea]. Disponible en: <http://arxiv.org/abs/1506.02640>
- [3] N. Kumar, R. Marée, P. Geurts, and M. Muller, “Recent advances in bioimage analysis methods for detecting skeletal deformities in biomedical and aquaculture fish species,” *Biomolecules*, vol. 13, no. 12, 2023. [En línea]. Disponible en: <https://www.mdpi.com/2218-273X/13/12/1797>
- [4] L. T. Ramos and A. D. Sappa, “A decade of you only look once (yolo) for object detection: A review,” *IEEE Access*, vol. 13, pp. 192 747–192 794, 2025.
- [5] S. R. Wenzhi Cao, Vahid Mirjalili, “Rank consistent ordinal regression for neural networks with application to age estimation,” *ScienceDirect*, 2020. [En línea]. Disponible en: https://www.sciencedirect.com/science/article/pii/S016786552030413X?via%3Dihub&__cf_chl_tk=aBmVdy36ROUaYAXk879UhzzoULjPmLCiMNx1YBSn2Ug-1775934534-1.0.1.1-HZqZuK1k_3vrCcpuyWCr823qFeehcV4NoclpM_QHPMY
- [6] Z. Niu, M. Zhou, L. Wang, X. Gao, and G. Hua, “Ordinal regression with multiple output cnn for age estimation,” in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 4920–4928.
- [7] T. T. Sundar, K. S. Vamsi, and G. S, “Deep learning-based plant disease detection and severity estimation: A comprehensive framework for precision agriculture,” in *2026 International Conference on Visual Analytics and Data Visualization (ICVADV)*, 2026, pp. 774–780.

- [8] S. Barrientos, C. Piñeiro-Corbeira, and R. Barreiro, “Temperate kelp forest collapse by fish herbivory: A detailed demographic study,” *Frontiers in Marine Science*, vol. 9, 2022. [En línea]. Disponible en: <https://www.frontiersin.org/journals/marine-science/articles/10.3389/fmars.2022.817021>
- [9] Ultralytics, “Models supported by ultralytics,” <https://docs.ultralytics.com/models>, 2023, documentación oficial de Ultralytics. consultado o 26 de maio de 2026.
- [10] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics yolov8,” 2023. [En línea]. Disponible en: <https://github.com/ultralytics/ultralytics>
- [11] M. Tan and Q. V. Le, “Efficientnet: Rethinking model scaling for convolutional neural networks,” *CoRR*, vol. abs/1905.11946, 2019. [En línea]. Disponible en: <http://arxiv.org/abs/1905.11946>
- [12] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” *CoRR*, vol. abs/1512.03385, 2015. [En línea]. Disponible en: <http://arxiv.org/abs/1512.03385>